

Building Up RL: From Dynamics and Control to Learning

PIERRE-LUC BACON

September 1, 2026

Contents

0.1	Syllabus u Plan de cours	1
0.2	English	1
0.2.1	Course information	1
0.2.2	Description	1
0.2.3	Prerequisites	1
0.2.4	Learning outcomes	2
0.2.5	Textbook	2
0.2.6	Evaluation	3
0.2.7	Tentative schedule	3
0.2.8	Course policies	5
0.3	Français	5
0.3.1	Renseignements généraux	5
0.3.2	Description	6
0.3.3	Préalables	6
0.3.4	Objectifs d'apprentissage	6
0.3.5	Manuel	7
0.3.6	Évaluation	7
0.3.7	Calendrier provisoire	8
0.3.8	Règlements du cours	10
0.4	Browser Labs	11
0.5	Modeling	12
0.5.1	Dynamics and State-Space Models	13
0.6	Numerical Trajectory Optimization	25
0.6.1	Discrete-Time Trajectory Optimization	26

0.1 Syllabus & Plan de cours

IFT 6162: Apprentissage par renforcement, commande optimale (Reinforcement Learning, Optimal Control)

Note

This page is bilingual. Use the tabs below to switch between English and French. Cette page est bilingue. Utilisez les onglets ci-dessous pour basculer entre l'anglais et le français.

0.2 English

0.2.1 Course information

Course code	IFT 6162
Title	Reinforcement Learning, Optimal Control (official registrar title is in French: <i>Apprentissage par renforcement, commande optimale</i>)
Credits	<i>TBD</i>
Term	Fall 2026
Schedule	Mondays 10:30–12:29 and Thursdays 13:30–15:29; Aug 31 – Oct 16 and Oct 26 – Dec 9, 2026 (14 teaching weeks, ~25 sessions)
Room	Campus Montréal, room <i>TBD</i>
Instructor	Pierre-Luc Bacon
Email	<i>TBD</i>
Office hours	<i>TBD</i>
Teaching assistants	<i>TBD</i>
Language of instruction	<i>TBD</i> (course materials are in English)
Course website	https://pierrelucbacon.com/rlbook/

0.2.2 Description

This course builds reinforcement learning up from its foundations in dynamics, optimization, and control. Rather than starting from the tabular Markov decision process and working outward, we start from the decision problem itself: how to write down a model of a system, how to state an objective that unfolds over time, and how to solve the resulting optimization problem numerically. Dynamic programming, model predictive control, and modern deep RL algorithms then appear as different answers to the same question, each carrying its own assumptions and failure modes.

The organizing theme is that most of the work in applying RL happens *before* an algorithm is chosen. Sensors produce noisy data, constraints are non-negotiable, and objectives shift or conflict. A practitioner who can only reach for a policy gradient method is poorly equipped for that reality. A practitioner who recognizes the same mathematical structure across trajectory optimization, MPC, dynamic programming, and deep RL can choose the right tool and explain why.

Reinforcement learning did not develop in isolation: its foundations draw on control theory, dynamic programming, operations research, and economics. The same ideas recur under different names in each community. Making those connections explicit is a central goal of the course.

0.2.3 Prerequisites

Students are expected to be comfortable with:

- **Mathematics.** Linear algebra, multivariable calculus, and probability at the undergraduate level.
- **Optimization.** Unconstrained and constrained optimization; gradient-based methods. Familiarity with KKT conditions is helpful but will be reviewed.
- **Programming.** Python, with working knowledge of NumPy. The course uses JAX and non-linear programming solvers; no prior experience with either is assumed.
- **Machine learning.** A prior course covering supervised learning and neural network training. Prior exposure to reinforcement learning is *not* required.

0.2.4 Learning outcomes

By the end of the course, you should be able to:

1. Formulate a sequential decision problem as a discrete-time optimal control problem or an MDP, stating dynamics, objective, and constraints explicitly, and justify the modeling choices you made.
2. Solve trajectory optimization problems numerically using single shooting, multiple shooting, and direct collocation, and explain the trade-offs among them in terms of conditioning, sparsity, and solver behavior.
3. Implement model predictive control, and reason about recursive feasibility, stability, constraint softening, and fallback behavior.
4. Derive dynamic programming recursions for finite- and infinite-horizon problems, and characterize Bellman operators as contractions on an appropriate space.
5. Explain approximate dynamic programming as the projection of a Bellman residual onto an approximation space, and connect this view to fitted value and fitted Q iteration.
6. Analyze the Monte Carlo estimators used throughout RL, identifying sources of bias and variance, including maximization bias.
7. Situate deep RL algorithms, including DQN and its extensions, DDPG, TD3, path consistency learning, MPPI, and policy gradient methods, within this framework, and explain each design choice as a response to a specific difficulty.
8. Read and critically assess research papers spanning the RL, control, and operations research literatures.

0.2.5 Textbook

The primary text is the course book, written for this class and freely available online:

Pierre-Luc Bacon. *Building Up RL: From Dynamics and Control to Learning*.
<https://pierre-luc-bacon.com/rlbook/>

The book is executable: figures and examples are generated from code you can run and modify. The source is on [GitHub](#); corrections and issues are welcome.

Supplementary references

These are recommended for depth on particular topics; none is required.

- D. P. Bertsekas. *Dynamic Programming and Optimal Control*, Vols. I–II. Athena Scientific.
- M. L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley, 1994.

- R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- J. T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*, 2nd ed. SIAM.
- J. B. Rawlings, D. Q. Mayne, and M. M. Diehl. *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill.
- J. Nocedal and S. J. Wright. *Numerical Optimization*, 2nd ed. Springer.
- W. B. Powell. *Reinforcement Learning and Stochastic Optimization*. Wiley, 2022.

0.2.6 Evaluation

Component	Weight	Due
Applied project	30%	<i>TBD</i>
In-class midterm 1	15%	<i>TBD</i>
In-class midterm 2	15%	<i>TBD</i>
In-class final examination	40%	<i>TBD</i>
Total	100%	

The three examinations are completed in person on paper, without electronic devices or generative-AI tools. Each student may bring one double-sided reference sheet that they prepared themselves. Questions may ask students to derive or interpret a result, inspect supplied code, identify a flaw in a model or algorithm, and justify a diagnosis. Memorizing Python syntax is not an examination objective.

Exercises, self-checks, and coding practice are formative and ungraded. They prepare students for the analytical and diagnostic work required on the examinations and in the project.

The applied project is completed in teams of exactly three. If enrollment makes that impossible, exceptions will be arranged privately by the instructor. Each team will formulate and investigate a substantive control-and-learning problem rather than reproduce a canned benchmark. The deliverables are working code, a poster, and an oral defense. A short proposal is an ungraded checkpoint. The team artifact establishes the base project grade; an individual's grade may be adjusted when the oral defense shows a materially different level of understanding.

Grading scale. *TBD*

Project checkpoint and late policy. Dates and late arrangements are *TBD*.

0.2.7 Tentative schedule

The course meets **twice a week**, Mondays 10:30–12:29 and Thursdays 13:30–15:29, over **14 teaching weeks**, for roughly 25 sessions in total. Each row below therefore covers two meetings, except where a holiday intervenes. Pacing is indicative and will be adjusted as the term goes on.

Week	Topic	Reading
1 ù Aug 31 – Sep 3	Course overview; system boundaries; model interfaces; state and action	Dynamics
2 ù Sep 7 – 10	Dynamics models and the state-space perspective	Dynamics
3 ù Sep 14 – 17	From deterministic to stochastic dynamics; partial observability; programs as models	Dynamics
4 ù Sep 21 – 24	Discrete-time optimal control problems; existence and optimality conditions	Trajectory Optimization
5 ù Sep 28 – Oct 1	Sequential and simultaneous methods; single and multiple shooting; adjoints as reverse accumulation	Trajectory Optimization
6 ù Oct 5 – 8	Direct transcription; polynomial interpolation	Collocation
7 ù Oct 12 – 15	A compendium of direct transcription methods; worked examples	Collocation
8 ù Oct 26 – 29	Model predictive control: closing the loop by replanning; theoretical guarantees; variants	MPC
9 ù Nov 2 – 5	MPC in practice: constraint softening, feasibility restoration, backup controllers; parametric optimization	MPC
10 ù Nov 9 – 12	Dynamic programming: backward recursion, continuous spaces, the linear quadratic regulator	Dynamic Programming
11 ù Nov 16 – 19	Markov decision processes: Bellman operators, infinite horizon, value and policy iteration	Dynamic Programming
12 ù Nov 23 – 26	Smoothing and regularized MDPs; projection and weighted residual methods	Smoothing, Projection
13 ù Nov 30 – Dec 3	Monte Carlo methods and overestimation bias; fitted Q iteration, NFQ, DQN	Monte Carlo, FQI
14 ù Dec 7	Amortized optimization (NFQCA, DDPG, TD3, PCL, MPPI); policy gradient methods	Amortization, Policy Gradients

No class on:

- Monday **September 7**: Labour Day (*Fête du travail*)
- Monday **October 12**: Thanksgiving (*Action de grâce*)
- **October 19–25**: *période d'activités libres* (no meetings; this is the gap between the two blocks in the registrar's listing)
- Thursday **December 10** falls outside the course's end date of December 9, so week 14 has a

Monday session only.

Note

The registrar lists the course as beginning **Monday, August 31**, while the University's academic calendar gives the *rentrée* as **Tuesday, September 1**. The date of the first Monday meeting will be confirmed before the term starts.

The appendices on [worked examples](#), [initial value problems](#), and [nonlinear programming](#) are reference material used throughout the term rather than assigned to a specific week.

0.2.8 Course policies

Attendance. *TBD*

Collaboration. Discussion and collaboration are encouraged on formative work. The project is collaborative within the assigned team. Each student must be able to explain and defend the team's complete submission.

Generative AI. Generative AI may be used for formative work and the applied project. Disclosure is not required. Each student remains responsible for every claim, equation, baseline, experiment, and line of submitted code and must be able to defend them during the oral defense. Generative AI and electronic devices are prohibited during in-class examinations.

Academic integrity. All work is subject to the University's regulations on plagiarism and fraud. See <https://integrite.umontreal.ca/> for the full policy.

Accommodations. Students registered with the *Soutien aux personnes étudiantes en situation de handicap* (SESH) service should contact the instructor early in the term so that arrangements can be made. See <https://vieetudiante.umontreal.ca/a-propos/service/soutien-personnes-etudiantes-situation-handicap> or write to soutienhandicap@sve.umontreal.ca.

Changes to this syllabus. This document may be revised during the term. Changes will be announced in class and reflected on this page.

0.3 Français

0.3.1 Renseignements généraux

Sigle	IFT 6162
Titre	Apprentissage par renforcement, commande optimale
Crédits	À déterminer
Trimestre	Automne 2026
Horaire	Lundi 10 h 30 – 12 h 29 et jeudi 13 h 30 – 15 h 29; du 31 août au 16 octobre et du 26 octobre au 9 décembre 2026 (14 semaines de cours, ~25 séances)
Local	Campus Montréal, local à déterminer
Enseignant	Pierre-Luc Bacon
Courriel	À déterminer
Disponibilités	À déterminer
Auxiliaires d'enseignement	À déterminer
Langue d'enseignement	À déterminer (le matériel du cours est en anglais)
Site du cours	https://pierrelucbacon.com/rlbook/

0.3.2 Description

Ce cours construit l'apprentissage par renforcement à partir de ses fondements en dynamique, en optimisation et en commande. Plutôt que de partir du processus décisionnel de Markov tabulaire pour élargir ensuite le cadre, nous partons du problème de décision lui-même : comment écrire le modèle d'un système, comment formuler un objectif qui se déploie dans le temps, et comment résoudre numériquement le problème d'optimisation qui en résulte. La programmation dynamique, la commande prédictive et les algorithmes modernes d'apprentissage par renforcement profond apparaissent alors comme autant de réponses à une même question, chacune portant ses propres hypothèses et ses propres modes de défaillance.

Le fil conducteur est que l'essentiel du travail d'application de l'apprentissage par renforcement se fait *avant* le choix d'un algorithme. Les capteurs produisent des données bruitées, certaines contraintes ne sont pas négociables, et les objectifs évoluent ou entrent en conflit. Une personne qui ne dispose que d'une méthode de gradient de politique est mal outillée devant cette réalité. Celle qui reconnaît la même structure mathématique dans l'optimisation de trajectoires, la commande prédictive, la programmation dynamique et l'apprentissage profond peut choisir le bon outil et justifier son choix.

L'apprentissage par renforcement ne s'est pas développé en vase clos : ses fondements puisent dans la théorie de la commande, la programmation dynamique, la recherche opérationnelle et l'économie. Les mêmes idées y reviennent sous des noms différents. Rendre ces liens explicites est un objectif central du cours.

0.3.3 Préalables

On attend des étudiantes et étudiants une aisance avec :

- **Mathématiques.** Algèbre linéaire, calcul à plusieurs variables et probabilités de niveau premier cycle.
- **Optimisation.** Optimisation avec et sans contraintes ; méthodes fondées sur le gradient. Une familiarité avec les conditions KKT est utile, mais celles-ci seront revues.
- **Programmation.** Python, avec une connaissance pratique de NumPy. Le cours utilise JAX et des solveurs de programmation non linéaire ; aucune expérience préalable n'est présumée pour l'un ou l'autre.
- **Apprentissage automatique.** Un cours antérieur couvrant l'apprentissage supervisé et l'entraînement de réseaux de neurones.

Aucune exposition préalable à l'apprentissage par renforcement n'est requise.

0.3.4 Objectifs d'apprentissage

Au terme du cours, vous devriez être en mesure de :

1. Formuler un problème de décision séquentielle comme un problème de commande optimale en temps discret ou comme un MDP, en énonçant explicitement la dynamique, l'objectif et les contraintes, et justifier vos choix de modélisation.
2. Résoudre numériquement des problèmes d'optimisation de trajectoires par tir simple, tir multiple et collocation directe, et expliquer les compromis entre ces approches en matière de conditionnement, de creux et de comportement du solveur.
3. Mettre en uvre la commande prédictive et raisonner sur la faisabilité récursive, la stabilité, l'assouplissement des contraintes et les comportements de repli.
4. Établir les récurrences de la programmation dynamique en horizon fini et infini, et caractériser les opérateurs de Bellman comme des contractions sur un espace approprié.
5. Expliquer la programmation dynamique approchée comme la projection d'un résidu de Bellman sur un espace d'approximation, et relier ce point de vue à l'itération sur la valeur ajustée et à l'itération sur Q ajustée.

6. Analyser les estimateurs de Monte-Carlo employés en apprentissage par renforcement en identifiant les sources de biais et de variance, dont le biais de maximisation.
7. Situer les algorithmes d'apprentissage profond, dont DQN et ses extensions, DDPG, TD3, l'apprentissage par cohérence de chemin, MPPI et les méthodes de gradient de politique, dans ce cadre, et expliquer chaque choix de conception comme une réponse à une difficulté précise.
8. Lire et évaluer de façon critique des articles de recherche issus des littératures de l'apprentissage par renforcement, de la commande et de la recherche opérationnelle.

0.3.5 Manuel

Le texte principal est le livre du cours, rédigé pour cette classe et accessible gratuitement en ligne :

Pierre-Luc Bacon. *Building Up RL: From Dynamics and Control to Learning*.
<https://pierrelucbacon.com/rlbook/>

Le livre est exécutable : les figures et les exemples sont produits à partir de code que vous pouvez exécuter et modifier. Le code source est sur [GitHub](#) ; les corrections et les signalements sont les bienvenus.

Références complémentaires

Recommandées pour approfondir certains sujets ; aucune n'est obligatoire.

- D. P. Bertsekas. *Dynamic Programming and Optimal Control*, vol. I–II. Athena Scientific.
- M. L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley, 1994.
- R. S. Sutton et A. G. Barto. *Reinforcement Learning: An Introduction*, 2^e éd. MIT Press, 2018.
- J. T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*, 2^e éd. SIAM.
- J. B. Rawlings, D. Q. Mayne et M. M. Diehl. *Model Predictive Control: Theory, Computation, and Design*, 2^e éd. Nob Hill.
- J. Nocedal et S. J. Wright. *Numerical Optimization*, 2^e éd. Springer.
- W. B. Powell. *Reinforcement Learning and Stochastic Optimization*. Wiley, 2022.

0.3.6 Évaluation

Élément	Pondération	Échéance
Projet appliqué	30 %	À déterminer
Premier examen de mi-session en classe	15 %	À déterminer
Deuxième examen de mi-session en classe	15 %	À déterminer
Examen final en classe	40 %	À déterminer
Total	100 %	

Les trois examens se font en personne, sur papier, sans appareil électronique ni outil d'intelligence artificielle générative. Chaque personne peut apporter une feuille de référence recto verso qu'elle a préparée elle-même. Les questions peuvent demander d'établir ou d'interpréter un résultat, d'examiner du code fourni, de repérer une erreur dans un modèle ou un algorithme et de justifier le diagnostic. La mémorisation de la syntaxe Python ne constitue pas un objectif d'évaluation.

Les exercices, les autoévaluations et les activités de programmation sont formatifs et ne sont pas notés. Ils préparent au travail d'analyse et de diagnostic demandé dans les examens et le projet.

Le projet appliqué se réalise en équipes d'exactly trois personnes. Si le nombre d'inscriptions rend cette règle impossible, l'enseignant réglera les exceptions en privé. Chaque équipe formulera et étudiera un problème substantiel de commande et d'apprentissage, plutôt que de reproduire un banc d'essai préfabriqué. Les livrables sont du code fonctionnel, une affiche et une défense orale. Une courte proposition sert de jalon non noté. La production d'équipe établit la note de base du projet; la note individuelle peut être ajustée si la défense orale indique un niveau de compréhension sensiblement différent.

Barème de notation. *À déterminer*

Jalon du projet et politique sur les retards. Les dates et les modalités sont *à déterminer*.

0.3.7 Calendrier provisoire

Le cours a lieu **deux fois par semaine**, le lundi de 10 h 30 à 12 h 29 et le jeudi de 13 h 30 à 15 h 29, sur **14 semaines de cours**, soit environ 25 séances au total. Chaque ligne ci-dessous couvre donc deux séances, sauf lorsqu'un congé s'intercale. Le rythme est indicatif et sera ajusté au fil du trimestre.

Semaine	Sujet	Lecture
1 û 31 août – 3 sept.	Présentation du cours ; frontière du système ; interfaces de modèle ; état et action	Dynamics
2 û 7 – 10 sept.	Modèles de dynamique et perspective de l'espace d'états	Dynamics
3 û 14 – 17 sept.	Du déterministe au stochastique ; observabilité partielle ; les programmes comme modèles	Dynamics
4 û 21 – 24 sept.	Problèmes de commande optimale en temps discret ; existence et conditions d'optimalité	Trajectory Optimization
5 û 28 sept. – 1 ^{er} oct.	Méthodes séquentielles et simultanées ; tir simple et tir multiple ; états adjoints et accumulation inverse	Trajectory Optimization
6 û 5 – 8 oct.	Transcription directe ; interpolation polynomiale	Collocation
7 û 12 – 15 oct.	Panorama des méthodes de transcription directe ; exemples détaillés	Collocation
8 û 26 – 29 oct.	Commande prédictive : boucler la boucle par replanification ; garanties théoriques ; variantes	MPC
9 û 2 – 5 nov.	La commande prédictive en pratique : assouplissement des contraintes, restauration de la faisabilité, contrôleurs de secours ; optimisation paramétrique	MPC
10 û 9 – 12 nov.	Programmation dynamique : récurrence arrière, espaces continus, régulateur linéaire quadratique	Dynamic Programming
11 û 16 – 19 nov.	Processus décisionnels de Markov : opérateurs de Bellman, horizon infini, itération sur la valeur et sur la politique	Dynamic Programming
12 û 23 – 26 nov.	Lissage et MDP régularisés ; méthodes de projection et de résidus pondérés	Smoothing, Projection
13 û 30 nov. – 3 déc.	Méthodes de Monte-Carlo et biais de surestimation ; itération sur Q ajustée, NFQ, DQN	Monte Carlo, FQI
14 û 7 déc.	Optimisation amortie (NFQCA, DDPG, TD3, PCL, MPPI) ; méthodes de gradient de politique	Amortization, Policy Gradients

Aucune séance :

- Lundi **7 septembre**: congé universitaire, Fête du travail
- Lundi **12 octobre**: congé universitaire, Action de grâce

- **19 au 25 octobre:** période d'activités libres (c'est l'intervalle entre les deux blocs de dates au répertoire des cours)
- Le jeudi **10 décembre** dépasse la date de fin du cours (9 décembre) : la semaine 14 ne comporte donc qu'une séance, le lundi.

Note

Le répertoire des cours indique un début le **lundi 31 août**, alors que le calendrier universitaire fixe la rentrée au **mardi 1^{er} septembre**. La date de la première séance du lundi sera confirmée avant le début du trimestre.

Les annexes sur les [exemples détaillés](#), les [problèmes à valeur initiale](#) et la [programmation non linéaire](#) servent de matériel de référence tout au long du trimestre plutôt que d'être rattachées à une semaine précise.

0.3.8 Règlements du cours

Présence. *À déterminer*

Collaboration. Les échanges et la collaboration sont encouragés dans les activités formatives. Le projet se réalise en collaboration au sein de l'équipe assignée. Chaque personne doit pouvoir expliquer et défendre l'ensemble de la production de son équipe.

Intelligence artificielle générative. L'intelligence artificielle générative peut être utilisée dans les activités formatives et le projet appliqué. Aucune déclaration d'utilisation n'est exigée. Chaque personne demeure responsable de chaque affirmation, équation, méthode de référence, expérience et ligne de code remise, et doit pouvoir les défendre lors de la défense orale. L'intelligence artificielle générative et les appareils électroniques sont interdits pendant les examens en classe.

Intégrité intellectuelle. Tous les travaux sont soumis au règlement disciplinaire de l'Université sur le plagiat et la fraude. Voir <https://integrite.umontreal.ca/> pour le texte complet.

Accommodements. Les personnes inscrites au service de Soutien aux personnes étudiantes en situation de handicap (SESH) sont invitées à communiquer avec l'enseignant tôt dans le trimestre afin que les mesures nécessaires soient mises en place. Voir <https://vieetudiante.umontreal.ca/a-propos/service/soutien-personnes-etudiantes-situation-handicap> ou écrire à soutienhandicap@sve.umontreal.ca.

Modifications au plan de cours. Ce document peut être révisé en cours de trimestre. Toute modification sera annoncée en classe et reflétée sur cette page.

0.4 Browser Labs

The browser lab is a small, install-free companion to the book. Its six notebooks focus on examples that fit comfortably in a WebAssembly Python kernel; the expensive JAX, MPC, and nonlinear solver demonstrations remain precomputed in the main text.

[Launch the browser labs](#)

Each notebook begins with a working baseline, asks you to predict an outcome before changing parameters, and ends with executable assertions:

1. finite-MDP policy evaluation;
2. value iteration and policy iteration;
3. Monte Carlo error and maximization bias;
4. Euler, trapezoidal, and higher-order ODE discretization;
5. finite-horizon LQR and the Riccati recursion;
6. a small SciPy trajectory-optimization problem.

The notebooks use only NumPy, SciPy, Matplotlib, and ipywidgets. Changes are stored in your browser. Download a notebook from JupyterLab if you want to keep or submit it.

To serve the lab locally after building it, run:

```
uv run jupyter lite serve - -lite -dir lab - -contents notebooks
```

0.5 Modeling

0.5.1 Dynamics and State-Space Models

A controller or learning algorithm acts on a description of a system. The description may be an equation, a simulator, a collection of transitions, or an interactive environment. Before choosing an algorithm, we need to determine what evolves, what can be changed, what can be observed, and which physical and operational restrictions must be preserved.

Learning Goals

After reading this chapter, you should be able to:

- define the state, action, disturbance, observation, objective, and constraints of a sequential decision problem;
- locate the physical channel through which an action changes a system;
- write deterministic and stochastic dynamics in continuous- and discrete-time state-space form;
- distinguish an open-loop action sequence from a feedback policy;
- classify the information supplied by equations, simulators, transition samples, and logged data;
- represent partial observability with an observation model;
- separate known structure from components that may be learned, and use trajectory diagnostics to detect a missing physical mode or constraint.

Prerequisites: Linear algebra, multivariable calculus, elementary probability, and the meaning of an ordinary differential equation. Numerical integration is reviewed in [Solving Initial Value Problems](#).

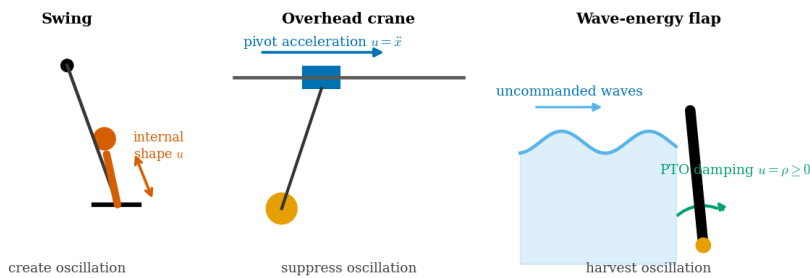
System Boundaries and Action Channels

A playground swing, an overhead crane, and a wave-energy converter all oscillate. Their visual similarity does not make them the same control problem. A rider changes body shape, a crane accelerates the suspension point, and a wave-energy converter changes a dissipative load while the sea supplies the forcing.

Formulation exercise

Before reading the table, predict which system should create oscillation, which should suppress it, and which should retain motion to extract energy. Then name the physical quantity that each controller can change.

Figure 1: The systems share an oscillatory silhouette but expose different control interfaces. The rider changes internal shape, the crane accelerates its pivot, and the wave device changes a nonnegative dissipative load. Their desired motions and feasible actions therefore differ.



system	state must describe	action changes	external influence	objective	hard constraint
playground swing	suspension angle and rider pose	internal body shape	gravity and drag	add mechanical energy	a chain can pull but cannot push
overhead crane	trolley position and load angle	trolley acceleration	gravity and damping	move the load and remove sway	acceleration is bounded
wave-energy converter	flap angle and velocity	power-take-off damping	incident waves	extract energy within motion limits	passive damping is nonnegative

Many mechanical models can be organized around a generalized position q , velocity v , auxiliary configuration z , and input u :

$$\dot{q} = v, \quad M(q, z)\dot{v} = -\nabla V(q, z) - D(q, v, z) + B(q, v, z)u + d(t). \quad (1)$$

The mass matrix M , potential V , dissipation D , actuation map B , and disturbance d have distinct physical roles. The term $B(q, v, z)u$ cannot be specified from the word “action” alone. An unconstrained vector input would omit the swing’s internal-actuation geometry, the crane’s acceleration limit, and the wave device’s passivity restriction.

The system boundary also determines which variables are external. A wave torque is a disturbance for the energy converter but would be an action in a wave-tank experiment. The boundary should follow the decision maker whose choices the model is intended to support.

Decision Models

In machine learning, a model often means a parameterized predictor. A **decision model** contains more: a state, available actions, dynamics, observations, an objective, constraints, and an information pattern. Prediction remains one component, but decisions require counterfactual trajectories under actions that may not yet have been observed.

For a finite horizon, one common formulation is

$$\begin{aligned} & \underset{u_0, \dots, u_{T-1}}{\text{minimize}} && \mathbb{E} \left[\sum_{t=0}^{T-1} \ell_t(x_t, u_t) + \ell_T(x_T) \right] \\ & \text{subject to} && x_{t+1} \sim P_t(\cdot \mid x_t, u_t), \\ & && u_t \in \mathcal{U}_t(x_t), \quad x_t \in \mathcal{X}_t. \end{aligned} \quad (2)$$

This expression says what is optimized, how uncertainty propagates, and which trajectories are admissible. Omitting one of these pieces changes the problem, even if the same environment class and neural network are used afterward.

An **open-loop plan** selects the whole action sequence from information available at the start,

$$(u_0, \dots, u_{T-1}) = \mu(x_0). \quad (3)$$

A **feedback policy** selects each action from the information available at that time,

$$u_t = \pi_t(I_t), \quad (4)$$

where I_t may contain the current state, observations, or a history. A planned trajectory can perform well under its assumed initial condition and disturbance forecast while reacting poorly to a tap, a delayed actuator, or an unexpected arrival. Later chapters turn open-loop plans into feedback by replanning and by computing state-contingent value functions.

Several independent choices describe the model itself.

axis	common alternatives	consequence
time	continuous or discrete	differential equations or transition steps
evolution	deterministic or stochastic	one next state or a distribution over next states
state	continuous, discrete, or mixed	geometry of the state space and applicable solvers
action	continuous, discrete, or mixed	control authority and optimization method
observation	full state or partial/noisy measurement	state feedback or estimation from information histories
horizon	finite, infinite, or terminating	terminal conditions and objective definition

Terms such as “continuous control” specify only one row of this table. They do not determine the time representation, uncertainty, observation model, or information available to the controller.

State-Space Models

A state is a summary of the past sufficient to predict the next state once the current action and disturbance are specified. It is defined relative to a model. A simulator may store a large internal state while exposing a smaller observation to the controller.

In discrete time, deterministic state-space dynamics take the form

$$x_{t+1} = f_t(x_t, u_t, d_t), \quad y_t = h_t(x_t, u_t, v_t), \quad (5)$$

where d_t denotes a process disturbance and v_t denotes measurement noise. In continuous time, the corresponding equations are

$$\dot{x}(t) = f(x(t), u(t), d(t)), \quad y(t) = h(x(t), u(t), v(t)). \quad (6)$$

Physical laws are often most legible in continuous time, while sensors, actuators, data sets, and software interfaces operate at discrete times. A numerical integrator and a sampling period connect the two descriptions. The sampling period is part of the model because changing it changes both the transition map and which disturbances can be resolved.

Linear continuous-time dynamics have the form

$$\dot{x} = Ax + Bu + Ed, \quad y = Cx + Du. \quad (7)$$

The matrix B describes how the action enters the dynamics, while C describes what is measured. These matrices encode different questions. Full actuation does not imply full observation, and full observation does not imply that every state component can be controlled.

Linearity is a modeling choice rather than a claim that the world is linear at all scales. A nonlinear system may be well approximated by a linear model near an equilibrium. [Dynamic Programming](#) uses that local approximation to stabilize an inverted pendulum, while [Trajectory Optimization](#) uses the nonlinear equations to move the pendulum into the local region.

Internal Actuation in SwingRL

The [swing-rl](#) environment asks whether a rider can pump a playground swing over the top bar without applying a motor torque at the pivot. Its articulated standing model exposes two normalized commands: squat depth and torso lean. A phase-dependent controller applies the squat command twice per swing cycle and the lean command once per cycle.

The first experiment asks whether these structured internal motions can produce a full rotation, and whether that conclusion survives a change from a rigid rod to a chain that cannot sustain compression. SwingRL supplies the multibody dynamics, unilateral chain simulator, rollout recorder, and renderer.

Figure 2: A standing-rider rollout generated by SwingRL’s articulated rigid-rod model. The controller changes squat depth twice per cycle and torso lean once per cycle. No torque is applied at the suspension point, and the displayed angle is unwrapped across complete rotations.

0

[x] Once [] Loop [] Reflect

Let θ be the suspension angle, ρ the rider’s center-of-mass distance from the pivot, α its offset from the suspension line, $\psi = \theta + \alpha$, β the body orientation relative to the suspension, and J the rider’s inertia about its center of mass. SwingRL’s reduced one-degree-of-freedom equation is

$$(m\rho^2 + J)\ddot{\theta} = -m\rho^2\ddot{\alpha} - 2m\rho\dot{\rho}\dot{\psi} - J\ddot{\beta} - J(\dot{\theta} + \dot{\beta}) - mg\rho\sin\psi + \tau_{\text{damp}}. \quad (8)$$

Torso lean contributes through the driven terms in $\ddot{\alpha}$ and $\ddot{\beta}$. Squatting changes ρ and contributes through the parametric term $-2m\rho\dot{\rho}\dot{\psi}$. During a forward bottom passage, $\dot{\psi} > 0$ and $\dot{\rho} < 0$, so this term is positive. During a backward passage both signs reverse, and the term again accelerates the current motion. The rider restores the longer configuration near a turning point, where $|\dot{\psi}|$ is smaller.

The experiment fixes the following model and evaluation settings.

quantity	value
suspension length	2.5 m
rider and seat mass	40 kg
center-of-mass inertia	2.0 kg m ²
integration interval	0.02 s
target angle	one complete unwrapped rotation, 2π rad
random seed	0

The standing and seated bodies use their matching hand-written controllers. The standing controller is also evaluated without modification on SwingRL’s chain model. The chain switches to ballistic motion when the suspension would require a negative force.

mechanism and suspension	outcome	peak angle	minimum suspension force
standing, rigid rod	full rotation at 26.04 s	360.30 degrees	-376 N
standing, chain	no rotation in 32 s	133.81 degrees	0 N
seated, rigid rod	no rotation in 60 s	48.92 degrees	307 N

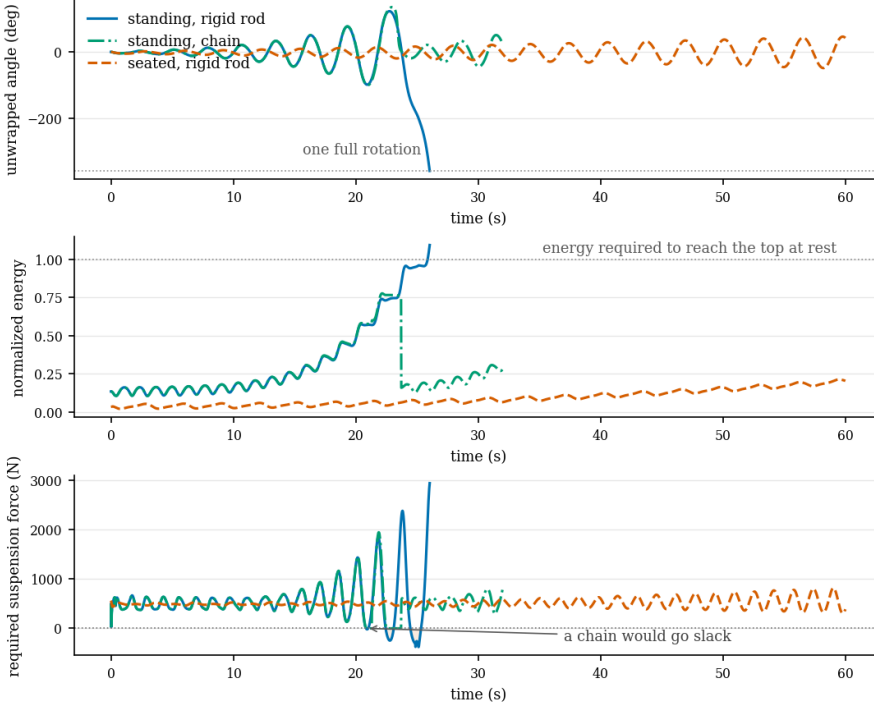
The rigid-rod result answers the original question only for a suspension that can both pull and push. An ordinary chain has two modes. In the taut mode, the constrained pendulum dynamics apply while the tension satisfies $T \geq 0$. At a release guard where the required tension reaches zero, the system enters a slack mode and follows ballistic dynamics. Contact with the chain-length boundary triggers a return to the taut mode, together with an impact law. This is a hybrid model because changing a parameter in the rod equation cannot represent the missing ballistic phase.

A learned approximation fitted only to rigid-rod trajectories would inherit the same failure. Data remain useful for estimating rider parameters, damping, and contact losses after the unilateral mode structure has been represented. [Policy Gradients](#) later compares the structured controller with a PPO policy trained on the same SwingRL plant and action limits.

Inference Serving as a Controlled System

An inference server receives prompts at irregular times and generates one response for each request. A decoder-only language model processes a request in two phases. The **prefill** phase processes the prompt in parallel and creates the first output token. The **decode** phase generates the remaining

Figure 3: The structured standing controller completes a rotation on the rigid-rod plant, but that trajectory eventually requires a negative suspension force. SwingRL’s unilateral chain model clips the feasible force at zero, enters a ballistic phase, and does not complete a rotation under the same controller. The seated rigid-rod trajectory remains taut but does not rotate within 60 seconds.



tokens one iteration at a time. Long prefills can delay the short decode iterations of requests already in progress. Iteration-level scheduling and chunked prefill are production approaches to controlling this interaction Yu et al. [2022], Agrawal et al. [2024]. The teaching simulator below uses a reduced interleaving rule rather than reproducing either serving system.

The system boundary surrounds one GPU and its serving process. The language model and inference engine lie inside the boundary; arriving requests and the ambient thermal conditions lie outside it. At a one-second control interval, a useful aggregate state is

$$x_t = (p_t, d_t, m_t, T_t, f_t), \quad (9)$$

where p_t is queued prefill work, d_t is unfinished decode work, m_t is key-value-cache occupancy, T_t is GPU temperature, and f_t is the realized graphics clock. The control

$$u_t = (f_t^{\text{req}}, \sigma_t) \quad (10)$$

combines a requested clock with a scheduling rule σ_t . The scheduler decides which phase receives each service step and how much prefill work may be processed before returning to active decodes. The clock request changes the rate and energy cost of that work. Hardware may realize a lower clock under a power or thermal limit, so f_t^{req} and f_t are distinct variables NVIDIA Corporation [2026].

Request arrivals form the disturbance. An arrival supplies its time and prompt length, but its eventual output length remains unknown to the controller until the end-of-sequence token arrives. The observation contains queue ages, completed prompt and output tokens, cache use, power, temperature, and realized clock. This information pattern prevents a controller from scheduling with the future length recorded in an evaluation trace.

The aggregate balances expose structure that does not have to be learned. If A_t^p prompt tokens arrive, C_t^p prompt tokens are processed, N_t^p requests finish prefill, and C_t^d decode tokens are produced, then

$$\begin{aligned} p_{t+1} &= p_t + A_t^p - C_t^p, \\ d_{t+1} &= d_t + W_t^d(N_t^p) - C_t^d. \end{aligned} \quad (11)$$

The term $W_t^d(N_t^p)$ denotes the still-unknown output work associated with newly admitted decode requests. The request-level simulator reveals this work only as tokens complete. It also maintains an exact cache allocation for every active sequence, so cache blocks are acquired after prefill and released at completion. An aggregate thermal balance has the form

$$T_{t+1} = T_t + \frac{\Delta t}{C_\theta} \left(P(x_t, u_t) - \frac{T_t - T_{\text{amb}}}{R_\theta} \right). \quad (12)$$

Token and cache conservation determine the form of the transition. Measurements are still needed for the service-rate map, the power map P , and the thermal parameters. The profiling protocol targets Qwen2.5-7B-Instruct served by vLLM on an NVIDIA L4 Qwen Team [2025b], Kwon et al. [2023]. Its intended hardware profile spans five requested clock levels, several prompt lengths, and three concurrency levels. The book build reads a committed profile and never starts a model server. Its manifest states whether the maps come from completed L4 measurements or a pre-measurement engineering surrogate, and every rendered result displays that provenance.

The workload is a five-minute excerpt from the Azure 2023 code-generation trace, which records request times and input and output token counts Microsoft Azure [2023]. Arrival times are dilated once to place maximum-clock utilization near 80 percent. If $\rho_{\max}$ is the isolated service time of all requests at the highest profiled clock divided by the original window length, the dilation and normalized arrivals are

$$d = \max\left(1, \frac{\rho_{\max}}{0.8}\right), \quad t'_i = d(t_i - t_0). \quad (13)$$

All controllers receive the same immutable requests after this transformation. The experimental time-to-first-token limit is twice the median baseline value for a 1,024-token prompt at concurrency one and the highest clock. The time-per-output-token limit is 1.5 times the corresponding median. These are reference levels for a controlled comparison, not service-level objectives claimed for production systems.

quantity	experiment setting
model	Qwen2.5-7B-Instruct, revision acbd96531cda22292a3ceaa67e984955d3965282 Qwen Team [2025a]
inference engine	vLLM OpenAI server, version 0.28.0 vLLM Project [2026]
target accelerator	one NVIDIA L4
clock profile	five requested graphics clocks; realized clock, power, utilization, and temperature retained
request trace	first five minutes for evaluation; first 20 requests for the animation
controller sampling	one second
service simulation	0.1-second steps; one phase per step; prefill chunks capped at 512 tokens
output length	hidden from the controller until completion

Within each one-second clock-control period, the scheduler serves either decode or one prefill chunk at each 0.1-second simulator step. When both phases have work, active decode receives alternating-step priority and strict priority under high cache pressure. This **512-token interleaved chunked-prefill** rule keeps the action channel visible without claiming to reproduce vLLM’s mixed-batch scheduler or Sarathi-Serve.

The first 20 requests form the animation below.

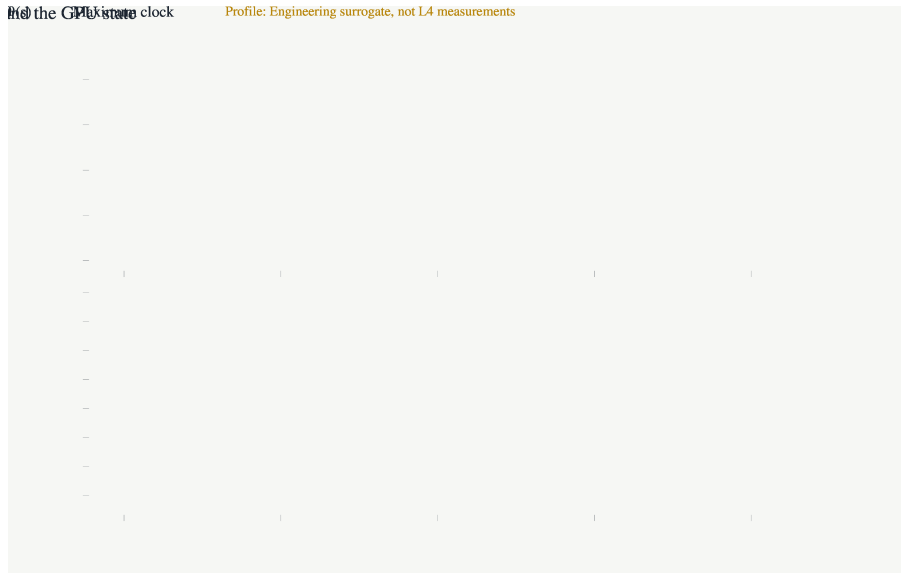


Figure 5: Static summary of unfinished requests and realized clock from the same request-level simulation. The online book provides playback, stepping, scrubbing, phase detail, and controller selection.

The experiment asks whether this state is sufficient to reproduce the service phenomena that matter for control. Conservation checks account for every request, processed token, and cache allocation. A concrete accounting check is visible at the end of the committed replay: all 20 requests have completed, the prefill and decode queues are empty, and all cache allocations have been released. This establishes closure of the modeled flows. It does not establish that the service-rate, power, or thermal maps are accurate. A provisional profile permits tests of the simulation and control software but does not supply empirical latency or energy evidence. In either case, the model does not cover every inference engine or GPU. Network transfer, host-side tokenization, multi-GPU communication, model-quality effects, and failures outside the serving process remain outside the boundary.

Stochastic Dynamics

Deterministic dynamics assign one next state to each state-action pair. Process noise, uncertain inflow, and unmodeled interactions instead produce a distribution of possible next states. A constructive representation makes the random input explicit:

$$x_{t+1} = f_t(x_t, u_t, w_t), \quad w_t \sim p_w. \quad (14)$$

For additive Gaussian noise, this becomes

$$x_{t+1} = Ax_t + Bu_t + w_t, \quad w_t \sim \mathcal{N}(0, Q). \quad (15)$$

The same stochastic dynamics can be represented directly by a transition kernel,

$$P_t(A \mid x, u) = \Pr(x_{t+1} \in A \mid x_t = x, u_t = u), \quad (16)$$

which assigns a probability to each measurable set of next states. The function-plus-noise representation exposes how randomness enters and may permit pathwise differentiation. The kernel representation requires only a distribution over next states and includes simulators whose internal random variables are hidden.

In continuous time, a stochastic differential equation separates drift and diffusion:

$$dX_t = f(X_t, U_t) dt + \sigma(X_t, U_t) dW_t. \quad (17)$$

The sampling interval again matters. A discrete model obtained from this equation must account for how diffusion accumulates over the interval rather than reusing the same noise covariance at every resolution.

Reservoir Conservation and Uncertain Inflow A hydroelectric reservoir retains a known conservation law even when inflow is uncertain. With stored volume x_t , turbine release u_t , spill s_t , and inflow w_t , a daily balance is

$$x_{t+1} = x_t + w_t - u_t - s_t. \quad (18)$$

Capacity and operational limits add

$$0 \leq x_t \leq x_{\max}, \quad 0 \leq u_t \leq u_{\max}, \quad s_t \geq 0. \quad (19)$$

Historical weather and flow records may supply an inflow distribution or a conditional predictor. They do not need to relearn that stored water equals previous storage plus inflow minus release and spill. The conservation law supports counterfactual release schedules, while the learned component captures the uncertain external driver.

Partial Observability

The state need not be directly measured. An observation model relates hidden state to sensor output:

$$y_t = h(x_t, u_t, v_t), \quad v_t \sim p_v. \quad (20)$$

For a linear Gaussian sensor, $y_t = Cx_t + v_t$ with $v_t \sim \mathcal{N}(0, R)$. A controller based only on y_t may lose information needed to predict future transitions. An estimator can instead summarize the observation history as a state estimate or a distribution over possible states.

Adaptive optics provides a concrete example. Atmospheric turbulence distorts the incoming optical wavefront, but a wavefront sensor measures noisy local slopes rather than the phase at every point. A deformable mirror applies the control input. A simple discrete model is

$$x_{t+1} = Ax_t + w_t, \quad y_t = Hx_t + v_t, \quad u_t = -K\hat{x}_t, \quad (21)$$

where x_t is the latent wavefront, H maps phase to measured slopes, and $\hat{x}_t$ is reconstructed from the measurement history. Treating y_t as the state would hide the reconstruction problem and can make the apparent dynamics non-Markovian.

Model Interfaces

Two models may generate the same nominal trajectory while exposing different operations to an algorithm. Access to equations, derivatives, resets, and new interactions determines what can be computed.

available interface	operations it supports directly	methods developed later
equations and derivatives	evaluate local dynamics, linearize, differentiate constraints	direct transcription, LQR, gradient-based MPC
one-step transition function	reset and advance from chosen state-action pairs	shooting, simulation-based MPC, model-based dynamic programming
generative simulator	sample trajectories without inspecting internal equations	Monte Carlo estimation, derivative-free search, policy learning
logged transitions	fit or evaluate models on the recorded distribution	system identification, fitted value and Q methods, offline evaluation
interactive environment and objective	collect new transitions under chosen actions	online reinforcement learning

The rows are not mutually exclusive. A simulator may be differentiable, an explicit model may contain unknown parameters, and logged data can be used to fit a new one-step model. The table records the information supplied to the algorithm, not the origin or fidelity of the model.

The inference case can be accessed through four of these interfaces without changing the system boundary. Token, cache, and thermal balances supply equations. The profile-based event simulator permits resets and counterfactual clock or scheduling actions. The trace and profiled transitions support only inferences on their recorded distribution. A live Qwen/vLLM process permits new requests under chosen clock settings. Moving down this list removes operations from the algorithm; it does not turn the underlying serving process into a different system.

A program is therefore a model when its execution defines state transitions. MuJoCo combines rigid-body equations, contact detection, constraint forces, sensors, and rendering. A user may have a reset-and-step interface without having a practical expression for its complete local transition function. Discrete-event simulators instead advance from one asynchronous event to the next. The inference-serving simulator advances through arrivals, completed prefill chunks, decode iterations, and request departures. It still defines a transition model, although its event clock and variable population of active requests differ from the fixed-step state-space models above.

Calling a method **model-free** describes an information restriction. The method does not query an explicit transition model during its update. It still depends on a specified state or observation, action set, reward, sampling process, and assumptions about how experience relates to future deployment.

Language Generation as a Sequential System Autoregressive language generation fits the same notation with an unusual transition. For a fixed prompt p , let

$$s_t = (p, y_1, \dots, y_{t-1}), \quad a_t = y_t. \quad (22)$$

The base transition appends the selected token,

$$s_{t+1} = \text{concat}(s_t, a_t). \quad (23)$$

This transition is deterministic once the token is chosen; uncertainty comes from the token policy $\pi(a_t | s_t)$. A language model therefore supplies a policy, while the prefix update supplies the transition. A reward or preference model is an additional object rather than an inherent property of text generation.

The simplified boundary changes when the model calls tools, receives new user messages, or interacts with an external application. Tool outputs and user responses then become observations generated by an environment outside the prefix concatenation rule. Context truncation also requires the retained context or memory state to be specified explicitly.

Known Structure and Learned Components

Uncertainty in one component does not require replacing the entire model. A structured transition can retain known dynamics and add a learned residual:

$$x_{t+1} = f_{\text{known}}(x_t, u_t) + r_{\theta}(x_t, u_t, z_t) + w_t. \quad (24)$$

The feature vector z_t may contain weather, payload information, or other measured conditions. The residual r_{θ} has a narrower task than a complete black-box transition: it predicts the part left unexplained by the known model.

system	retained structure	plausible learned component	model-audit experiment
swing	internal actuation and unilateral tension	rider, damping, and contact-loss parameters	measure the first slack event
reservoir	water conservation and capacity limits	conditional inflow distribution	test across seasons and extreme precipitation
inference service	request, token, and cache conservation	service-rate, power, and thermal residuals	hold out prompt lengths, clocks, and concurrency levels

Generative models can produce plausible simulator and controller code quickly. That code does not establish that a chain can sustain the simulated force, that an action has the intended physical meaning, or that a scalar reward contains a safety requirement. Those claims require inspection of the model and evidence from targeted experiments.

A residual cannot repair a missing action channel or a missing physical mode if the training data never contains evidence of it. The SwingRL comparison exposed the failure by changing the suspension model while holding the controller fixed. Comparable interventions are needed to decide which structure should remain and which component should be learned.

Summary

A decision model specifies state, action, dynamics, observations, objective, constraints, time, and information. The state-space representation separates what evolves from what is observed and what can be controlled. Continuous and discrete time, deterministic and stochastic evolution, and full and partial observation are independent modeling choices.

The SwingRL experiment showed a structured controller completing a rotation on a rigid rod and failing on a unilateral chain. The discrepancy was caused by a missing mode, not by an uncertain coefficient. The inference and reservoir examples showed the complementary case, where conservation laws can be retained while service rates, thermal parameters, or disturbances are estimated from data.

The model interface determines the methods available to the decision maker. The next chapter assumes a dynamics model that can generate and constrain candidate trajectories, then computes an open-loop plan. Later chapters add feedback and learn model components, values, or policies when the corresponding objects are unavailable or too costly to compute directly.

Exercises

1. For an unfamiliar system, write five lines specifying its state, action, disturbance, objective, and one hard constraint. State where the system boundary is drawn.
2. In the action-channel figure, explain why replacing every input by an unconstrained generalized force changes all three physical problems.
3. Convert $\dot{x} = -ax + bu$ to a discrete-time model using forward Euler with step size Δt . Identify the resulting coefficients.
4. Use the sign of $-2m\rho\dot{\psi}$ to explain why shortening the rider's radius near the bottom can add energy on both half-cycles.
5. Express the swing's taut and slack phases as a hybrid system. Identify the continuous state, discrete mode, release guard, and reattachment event.
6. The structured swing controller succeeds on the rod and fails on the chain. Explain why fitting a larger neural transition model to rod-only data does not, by itself, resolve this discrepancy.

7. For the inference service, write a cache-balance equation that distinguishes blocks allocated at prefill completion from blocks released at request completion. State one condition that makes the aggregate cache state insufficient for prediction.
8. Write the reservoir transition as a function-plus-noise model when inflow is $w_t = \bar{w}_t + \epsilon_t$. Describe the corresponding transition kernel.
9. Give an example where the observation is not a Markov state. State which missing variable or history would improve prediction.
10. Classify each of the following interfaces: a differentiable ODE solver, a reset-and-step robotics simulator, and a fixed table of logged transitions. Name one method from the book that each interface supports directly.
11. For autoregressive language generation, distinguish the environment transition from the token policy. Explain how the model changes when a tool call returns information from an external database.
12. Run [the SwingRL control example](#) and change one rider or damping parameter. Before running it, predict which trajectory diagnostic will change. Report whether the result supports the prediction and which assumption the experiment tests.
13. Load the Azure trace subset used by the inference experiment. Recompute the offered-load dilation after omitting the ten longest prompts. Explain which model quantity changes and which conservation equations remain unchanged.

Computational Sources

The SwingRL dependency is pinned to commit [d579663](#). The chapter executes four downloadable source files:

- [SwingRL controller and model audit](#)
- [Action-channel figure](#)
- [Inference-serving model](#)
- [Inference replay renderer](#)

The animation controls are embedded in the static site and require no live Python kernel.

Self-checks

Exercise 1: State or observation?

An inference controller observes queue counts and cache occupancy but not the remaining output length of each active request. Are those measurements alone a Markov state for the request-level process? Explain briefly.

Solution to Exercise 1: State or observation?

Generally no. Two systems can have the same counts and cache occupancy while their active requests have different remaining output lengths, which changes their future completion times and cache release.

Exercise 2: Discretization check

For $\dot{x} = ax + bu$ with a zero-order-held input and step Δt , write the explicit Euler update.

Solution to Exercise 2: Discretization check

$$x_{k+1} = x_k + \Delta t(ax_k + bu_k) = (1 + a\Delta t)x_k + b\Delta t u_k.$$

Exercise 3: Where uncertainty belongs

When is a transition kernel more natural than writing deterministic dynamics plus additive noise?

Solution to Exercise 3: Where uncertainty belongs

A kernel is more natural when uncertainty is discrete, state dependent, multimodal, or otherwise cannot be represented faithfully as a simple additive disturbance.

0.6 Numerical Trajectory Optimization

0.6.1 Discrete-Time Trajectory Optimization

In the previous chapter, we examined different ways to represent dynamical systems: continuous versus discrete time, deterministic versus stochastic, fully versus partially observable, and even simulation-based views such as agent-based or programmatic models. Our focus was on the **structure of models**: how they capture evolution, uncertainty, and information.

In this chapter, we turn to what makes these models useful for **decision-making**. The goal is no longer just to describe how a system behaves, but to leverage that description to **compute actions over time**. This doesn't mean the model prescribes actions on its own. Rather, it provides the scaffolding for optimization: given a model and an objective, we can derive the control inputs that make the modeled system behave well according to a chosen criterion.

Our entry point will be trajectory optimization. By a **trajectory**, we mean the time-indexed sequence of states and controls that the system follows under a plan: the states $(\mathbf{x}_1, \dots, \mathbf{x}_T)$ together with the controls $(\mathbf{u}_1, \dots, \mathbf{u}_{T-1})$. In this chapter, we focus on an **open-loop** viewpoint: starting from a known initial state, we compute the entire sequence of controls in advance and then apply it as-is. This is appealing because, for discrete-time problems, it yields a finite-dimensional optimization over a vector of decisions and cleanly exposes the structure of the constraints. In continuous time, the base formulation is infinite-dimensional; in this course we will rely on direct methods (time discretization and parameterization) to transform it into a finite-dimensional nonlinear program.

Open loop also has a clear limitation: if reality deviates from the model, whether due to disturbances, model mismatch, or unanticipated events, the state you actually reach may differ from the predicted one. The precomputed controls that were optimal for the nominal trajectory can then lead you further off course, and errors can compound over time.

Later, we will study **closed-loop (feedback)** strategies, where the choice of action at time t can depend on the state observed at time t . Instead of a single sequence, we optimize a policy π_t mapping states to controls, $\mathbf{u}_t = \pi_t(\mathbf{x}_t)$. Feedback makes plans resilient to unforeseen situations by adapting on the fly, but it leads to a more challenging problem class. We start with open-loop trajectory optimization to build core concepts and tools before tackling feedback design.

Learning Goals

After studying this chapter, you should be able to:

1. Formulate a discrete-time optimal control problem (DOCP) in Bolza, Lagrange, and Mayer forms, and convert between them.
2. State the KKT conditions for a constrained NLP and explain the role of constraint qualifications.
3. Derive the discrete-time Pontryagin principle from the Lagrangian of a DOCP.
4. Implement single shooting and multiple shooting methods for trajectory optimization.
5. Explain the trade-offs between simultaneous (direct transcription) and sequential (shooting) methods.
6. Compute gradients of the objective with respect to controls using the adjoint (costate) recursion.

Prerequisites

This chapter assumes familiarity with:

- Multivariable calculus (gradients, Jacobians, chain rule)
- Basic optimization (unconstrained minimization, gradient descent)
- Linear algebra (matrix-vector products, linear systems)

- Dynamical systems from the previous chapter

A Motivating Example: Inventory Control Before diving into the general formulation, consider a concrete problem that arises in operations management. A retailer must decide how much inventory to order at the start of each period to meet uncertain demand while balancing holding costs against stockout penalties.

The setup is simple. At the beginning of period k , the retailer observes the current inventory level x_k and places an order of size $u_k \geq 0$. Demand d_k then arrives (assume for now it is known in advance), and the inventory evolves according to

$$x_{k+1} = x_k + u_k - d_k. \quad (25)$$

If inventory is positive at the end of the period, holding costs accrue. If inventory is negative (backorders), penalty costs apply. There may also be a per-unit ordering cost. A typical objective is to minimize total cost over a planning horizon of T periods:

$$\min_{u_0, \dots, u_{T-1}} \sum_{k=0}^{T-1} (h[x_k]_+ + p[-x_k]_+ + c u_k), \quad (26)$$

where $[z]_+ = \max(0, z)$, h is the holding cost rate, p is the backorder penalty rate, and c is the ordering cost per unit. The retailer may also face constraints: orders cannot be negative, and inventory might be bounded above by warehouse capacity.

This problem has a clear structure: a state (x_k) that evolves according to a known rule, a control (u_k) that influences the next state, a cost that accumulates over time, and constraints on the decisions. These ingredients define a **discrete-time optimal control problem** (DOCP). We now formalize this structure in general terms.

Discrete-Time Optimal Control Problems (DOCPs) Consider a system described by a **state** $\mathbf{x}_t \in \mathbb{R}^n$, summarizing everything needed to predict its evolution. At each stage t , we can influence the system through a **control input** $\mathbf{u}_t \in \mathbb{R}^m$. The dynamics specify how the state evolves:

$$\mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t), \quad (27)$$

where $\mathbf{f}_t$ may be nonlinear or time-varying. We assume the initial state $\mathbf{x}_1$ is known.

The goal is to pick a sequence of controls $\mathbf{u}_1, \dots, \mathbf{u}_{T-1}$ that makes the trajectory desirable. But desirable in what sense? That depends on an **objective function**, which often includes two components:

$$(i) \text{ stage cost: } c_t(\mathbf{x}_t, \mathbf{u}_t), \quad (ii) \text{ terminal cost: } c_T(\mathbf{x}_T). \quad (28)$$

The stage cost reflects ongoing penalties such as energy, delay, or risk. The terminal cost measures the value (or cost) of ending in a particular state. Together, these give a discrete-time Bolza problem with path constraints and bounds:

$$\begin{aligned} \text{minimize} \quad & c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \\ \text{subject to} \quad & \mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) \\ & \mathbf{g}_t(\mathbf{x}_t, \mathbf{u}_t) \leq \mathbf{0} \\ & \mathbf{x}_{\min} \leq \mathbf{x}_t \leq \mathbf{x}_{\max} \\ & \mathbf{u}_{\min} \leq \mathbf{u}_t \leq \mathbf{u}_{\max} \\ \text{given} \quad & \mathbf{x}_1 = \mathbf{x}_0. \end{aligned} \quad (29)$$

Returning to the inventory example, the correspondence is direct: the state $\mathbf{x}_t$ is the inventory level x_k , the control $\mathbf{u}_t$ is the order quantity u_k , and the dynamics $\mathbf{f}_t$ encode the inventory balance equation $x_{k+1} = x_k + u_k - d_k$. The stage cost c_t captures holding, backorder, and ordering costs, while the terminal cost c_T might penalize ending with excess stock. Constraints $\mathbf{g}_t \leq \mathbf{0}$ can enforce non-negativity of orders or warehouse capacity limits. This mapping shows how practical problems fit naturally into the abstract framework.

Written this way, it may seem obvious that the decision variables are the controls $\mathbf{u}_t$. After all, in most intuitive descriptions of control, we think of choosing inputs to influence the system. But notice that in the program above, the entire state trajectory also appears as a set of variables, linked to the controls by the dynamics constraints. This is intentional: it reflects one way of writing the problem that makes the constraints explicit.

Why introduce $\mathbf{x}_t$ as decision variables if they can be simulated forward from the controls? Many readers hesitate here, and the question is natural: *If the model is deterministic and $\mathbf{x}_1$ is known, why not pick $\mathbf{u}_{1:T-1}$ and compute $\mathbf{x}_{2:T}$ on the fly?* That instinct leads to **single shooting**, a method we will return to shortly.

Already in this formulation, though, **the structure of the problem matters**. Ignoring it can make our life much harder. The reason is twofold:

- **Dimensionality grows with the horizon.** For a horizon of length T , the program has roughly $(T-1)(m+n)$ decision variables.
- **Temporal coupling.** Each control affects all future states and costs. The feasible set is not a simple box but a narrow manifold defined by the dynamics.

Together, these features explain why specialized methods exist and why the way we write the problem influences the algorithms we can use. Whether we keep states explicit or eliminate them through forward simulation determines the problem size, its conditioning, and the trade-offs between robustness and computational effort.

Existence of Solutions and Optimality Conditions Now that we have the optimization problem written down, we can ask: does it always have a solution? And if so, how do we recognize one? These questions lead us to feasibility and optimality conditions.

Existence of Solutions Notice first that nothing in the problem statement required the dynamics

$$\mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) \quad (30)$$

to be stable. In fact, many problems of interest involve unstable systems; think of balancing a pole or steering a spacecraft. What matters is that the dynamics are **well defined**: given a state-control pair, the rule $\mathbf{f}_t$ produces a valid next state.

In continuous time, one usually requires $\mathbf{f}$ to be continuous (often Lipschitz continuous) in $\mathbf{x}$ so that the ODE has a unique solution on the horizon of interest. In discrete time, the requirement is lighter: we only need the update map to be well posed.

Existence also hinges on **feasibility**. A candidate control sequence must generate a trajectory that respects all constraints: the dynamics, any bounds on state and control, and any terminal requirements. If no such sequence exists, the feasible set is empty and the problem has no solution. This can happen if the constraints are overly strict, or if the system is uncontrollable from the given initial condition.

Optimality Conditions Assume the feasible set is nonempty. To characterize a point that is not only feasible but **locally optimal**, we use the Lagrange multiplier machinery from nonlinear programming. For a smooth problem

$$\begin{aligned}
& \min_{\mathbf{z}} && F(\mathbf{z}) \\
& \text{s.t.} && G(\mathbf{z}) = \mathbf{0}, \\
& && H(\mathbf{z}) \geq \mathbf{0},
\end{aligned} \tag{31}$$

define the **Lagrangian**

$$\mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = F(\mathbf{z}) + \boldsymbol{\lambda}^\top G(\mathbf{z}) + \boldsymbol{\mu}^\top H(\mathbf{z}), \quad \boldsymbol{\mu} \geq \mathbf{0}. \tag{32}$$

For an inequality system $H(\mathbf{z}) \geq \mathbf{0}$ and a candidate point $\mathbf{z}$, the **active set** is

$$\mathcal{A}(\mathbf{z}) = \{i : H_i(\mathbf{z}) = 0\}, \tag{33}$$

while indices with $H_i(\mathbf{z}) > 0$ are **inactive**. Only active inequalities can carry positive multipliers.

We now make a **constraint qualification** assumption. In plain language, it says the constraints near the solution intersect in a regular way so that the feasible set has a well-defined tangent space and the multipliers exist. Algebraically, this amounts to a **full row rank** condition on the Jacobian of the equalities together with the active inequalities:

$$\text{rows of } [\nabla G(\mathbf{z}^*); \nabla H_{\mathcal{A}}(\mathbf{z}^*)] \text{ are linearly independent.} \tag{34}$$

This is the **LICQ** (Linear Independence Constraint Qualification). In convex problems, **Slater's condition** (existence of a strictly feasible point) plays a similar role. You can think of these as the assumptions that let the linearized KKT equations be solvable; we do not literally invert that Jacobian, but the full-rank property is what would make such an inversion possible in principle.

Under such a constraint qualification, any local minimizer $\mathbf{z}^*$ admits multipliers $(\boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ that satisfy the **Karush–Kuhn–Tucker (KKT) conditions**:

$$\begin{aligned}
& \text{stationarity:} && \nabla_{\mathbf{z}} \mathcal{L}(\mathbf{z}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) = \mathbf{0}, \\
& \text{primal feasibility:} && G(\mathbf{z}^*) = \mathbf{0}, \quad H(\mathbf{z}^*) \geq \mathbf{0}, \\
& \text{dual feasibility:} && \boldsymbol{\mu}^* \geq \mathbf{0}, \\
& \text{complementarity:} && \mu_i^* H_i(\mathbf{z}^*) = 0 \quad \text{for all } i.
\end{aligned} \tag{35}$$

Only constraints that are **active** at $\mathbf{z}^*$ can have $\mu_i^* > 0$; inactive ones have $\mu_i^* = 0$. The multipliers quantify marginal costs: λ_j^* measures how the optimal value changes if the j -th equality is relaxed, and μ_i^* does the same for the i -th inequality. (If you prefer $h(\mathbf{z}) \leq 0$, signs flip accordingly.)

In our trajectory problems, $\mathbf{z}$ stacks state and control trajectories, G enforces the dynamics, and H collects bounds and path constraints. The equalities' multipliers act as **costates** or **shadow prices** for the dynamics. Writing the KKT system stage by stage yields the discrete-time Pontryagin principle, derived next. For convex programs these conditions are also sufficient.

What fails without a CQ? If the active gradients are dependent (for example duplicated or nearly parallel), the Jacobian loses rank; multipliers may then be nonunique or fail to exist, and the linearized equations become ill-posed. In transcribed trajectory problems this shows up as dependent dynamic constraints or redundant path constraints, which leads to fragile solver behavior.

The KKT conditions provide necessary conditions for a point to be a local minimizer of a constrained optimization problem. They consist of four parts: stationarity (the gradient of the Lagrangian vanishes), primal feasibility (constraints are satisfied), dual feasibility (inequality multipliers are non-negative), and complementarity (inactive constraints have zero multipliers). The multipliers have an economic interpretation as marginal costs: they tell us how much the optimal value would change if we relaxed a constraint slightly. For convex problems, the KKT conditions are also sufficient, meaning any point satisfying them is globally optimal. In trajectory optimization, these conditions will reappear in structured form as the Pontryagin principle.

From KKT to algorithms The KKT system can be read as the first-order optimality conditions of a **saddle-point** problem. With equalities $G(\mathbf{z}) = \mathbf{0}$ and inequalities $H(\mathbf{z}) \geq \mathbf{0}$, define the Lagrangian

$$\mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = F(\mathbf{z}) + \boldsymbol{\lambda}^\top G(\mathbf{z}) + \boldsymbol{\mu}^\top H(\mathbf{z}), \quad \boldsymbol{\mu} \geq \mathbf{0}. \quad (36)$$

Optimality corresponds to a saddle: minimize in $\mathbf{z}$, maximize in $(\boldsymbol{\lambda}, \boldsymbol{\mu})$ (with $\boldsymbol{\mu}$ constrained to the nonnegative orthant).

Primal–dual gradient dynamics (Arrow–Hurwicz) The simplest algorithm mirrors this saddle structure by descending in the primal variables and ascending in the dual variables, with a projection for the inequalities:

$$\begin{aligned} \mathbf{z}^{k+1} &= \mathbf{z}^k - \alpha_k (\nabla F(\mathbf{z}^k) + \nabla G(\mathbf{z}^k)^\top \boldsymbol{\lambda}^k + \nabla H(\mathbf{z}^k)^\top \boldsymbol{\mu}^k), \\ \boldsymbol{\lambda}^{k+1} &= \boldsymbol{\lambda}^k + \beta_k G(\mathbf{z}^k), \\ \boldsymbol{\mu}^{k+1} &= \Pi_{\geq 0}(\boldsymbol{\mu}^k + \beta_k H(\mathbf{z}^k)). \end{aligned} \quad (37)$$

Here $\Pi_{\geq 0}$ is the projection onto $\{\boldsymbol{\mu} \geq \mathbf{0}\}$. In convex settings and with suitable step sizes, these iterates converge to a saddle point. In nonconvex problems (our trajectory optimizations after transcription), these updates are often used inside **augmented Lagrangian** or **penalty** frameworks to improve robustness, for example by replacing $\mathcal{L}$ with

$$\mathcal{L}_\rho(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\mu}) + \frac{\rho}{2} \|G(\mathbf{z})\|^2 + \frac{\rho}{2} \|\min\{0, H(\mathbf{z})\}\|^2, \quad (38)$$

which stabilizes the dual ascent when constraints are not yet well satisfied.

SQP as Newton on the KKT system (equality case) With only equality constraints $G(\mathbf{z}) = \mathbf{0}$, write first-order conditions

$$\nabla_{\mathbf{z}} \mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}) = \mathbf{0}, \quad G(\mathbf{z}) = \mathbf{0}, \quad \text{where } \mathcal{L} = F + \boldsymbol{\lambda}^\top G. \quad (39)$$

Applying Newton’s method to this system gives the linear KKT solve

$$\begin{bmatrix} \nabla_{\mathbf{z}\mathbf{z}}^2 \mathcal{L}(\mathbf{z}^k, \boldsymbol{\lambda}^k) & \nabla G(\mathbf{z}^k)^\top \\ \nabla G(\mathbf{z}^k) & 0 \end{bmatrix} \begin{bmatrix} \Delta \mathbf{z} \\ \Delta \boldsymbol{\lambda} \end{bmatrix} = - \begin{bmatrix} \nabla_{\mathbf{z}} \mathcal{L}(\mathbf{z}^k, \boldsymbol{\lambda}^k) \\ G(\mathbf{z}^k) \end{bmatrix}. \quad (40)$$

This is exactly the step computed by **Sequential Quadratic Programming (SQP)** in the equality-constrained case: it is Newton’s method on the KKT equations. For general problems with inequalities, SQP forms a **quadratic subproblem** by quadratically modeling F with $\nabla_{\mathbf{z}\mathbf{z}}^2 \mathcal{L}$ and linearizing the constraints, then solves that QP with line search or trust region. In least-squares-like problems one often uses **Gauss–Newton** (or a Levenberg–Marquardt trust region) as a positive-definite approximation to the Lagrangian Hessian.

In trajectory optimization, the KKT matrix inherits banded/sparse structure from the dynamics. Newton/SQP steps can be computed efficiently by exploiting this structure; in the special case of quadratic models and linearized dynamics, the QP reduces to an LQR solve along the horizon (this is the backbone of iLQR/DDP-style methods). Primal-dual updates provide simpler iterations and are easy to implement; augmented terms are typically needed to obtain stable progress when constraints couple stages.

The choice between methods depends on the context. Primal-dual gradients give lightweight iterations and are suited for warm starts or as inner loops with penalties. SQP/Newton gives rapid local convergence when close to a solution and LICQ holds; trust regions or line search help globalize convergence.

Examples of DOCPs To make things concrete, here are additional problems that are naturally posed as discrete-time OCPs. We already introduced inventory control at the start of this chapter; below are two more examples from different domains, followed by a discussion of how continuous-time problems give rise to DOCPs through discretization.

End-of-Day Portfolio Rebalancing At each trading day k , choose trades u_k to adjust holdings h_k before next-day returns r_k realize. Deterministic planning uses predicted returns μ_k , with dynamics $h_{k+1} = (h_k + u_k) \odot (\mathbf{1} + \mu_k)$ and budget/box constraints. The stage cost can capture transaction costs and risk, e.g., $c_k(h_k, u_k) = \tau \|u_k\|_1 + \frac{\lambda}{2} h_k^\top \Sigma_k h_k$, with a terminal utility or wealth objective.

Daily Ad-Budget Allocation with Carryover Allocate spend $u_k \in [0, U_{\max}]$ to build awareness s_k with carryover dynamics $s_{k+1} = \alpha s_k + \beta u_k$. Conversions/revenue at day k follow a response curve $g(s_k, u_k)$; the goal is $\max \sum_{k=0}^{T-1} g(s_k, u_k) - c u_k$ subject to spend limits. This is naturally discrete because decisions and measurements occur daily.

DOCPs Arising from the Discretization of Continuous-Time OCPs Although many applications are natively discrete-time, it is also common to obtain a DOCP by discretizing a continuous-time formulation. Consider a system on $[0, T_c]$ given by

$$\dot{\mathbf{x}}(t) = \mathbf{f}(t, \mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{x}(0) = \mathbf{x}_0. \quad (41)$$

Choose a step size $\Delta > 0$ and grid $t_k = k \Delta$. A one-step integration scheme induces a discrete map $\mathbf{F}_\Delta$ so that

$$\mathbf{x}_{k+1} = \mathbf{F}_\Delta(\mathbf{x}_k, \mathbf{u}_k, t_k), \quad k = 0, \dots, T-1, \quad (42)$$

where, for example, explicit Euler gives $\mathbf{F}_\Delta(\mathbf{x}, \mathbf{u}, t) = \mathbf{x} + \Delta \mathbf{f}(t, \mathbf{x}, \mathbf{u})$. The resulting discrete-time optimal control problem takes the Bolza form with these induced dynamics:

$$\begin{aligned} \min_{\{\mathbf{x}_k, \mathbf{u}_k\}} \quad & c_T(\mathbf{x}_T) + \sum_{k=0}^{T-1} c_k(\mathbf{x}_k, \mathbf{u}_k) \\ \text{s.t.} \quad & \mathbf{x}_{k+1} - \mathbf{F}_\Delta(\mathbf{x}_k, \mathbf{u}_k, t_k) = 0, \quad k = 0, \dots, T-1, \\ & \mathbf{x}_0 = \mathbf{x}_{\text{init}}. \end{aligned} \quad (43)$$

Programs as DOCPs and Differentiable Programming It is often useful to view a computer program itself as a discrete-time dynamical system. Let the **program state** collect memory, buffers, and intermediate variables, and let the **control** represent inputs or tunable decisions at each step. A single execution step defines a transition map

$$\mathbf{x}_{k+1} = \Phi_k(\mathbf{x}_k, \mathbf{u}_k), \quad (44)$$

and a scalar objective (e.g., loss, error, runtime, energy) yields a DOCP:

$$\min_{\{\mathbf{u}_k\}} c_T(\mathbf{x}_T) + \sum_{k=0}^{T-1} c_k(\mathbf{x}_k, \mathbf{u}_k) \quad \text{s.t.} \quad \mathbf{x}_{k+1} = \Phi_k(\mathbf{x}_k, \mathbf{u}_k). \quad (45)$$

In differentiable programming (e.g., JAX, PyTorch), the composed map $\Phi_{T-1} \circ \dots \circ \Phi_0$ is differentiable, enabling reverse-mode automatic differentiation and efficient gradient-based trajectory optimization. When parts of the program are non-differentiable (discrete branches, simulators with events), DOCPs can still be solved using derivative-free or weak-gradient methods (eg. finite differences, SPSA, Nelder–Mead, CMA-ES, or evolutionary strategies) optionally combined with smoothing, relaxations, or stochastic estimators to navigate non-smooth regions.

Example: Offline Frequency Planning for Inference The inference service from the modeling chapter is also a program with a controllable execution rate. The scheduling rule is held fixed here. Each 0.1-second simulator step serves one phase, prefill work is capped at 512 tokens per prefill step, and active decode receives alternating-step or cache-pressure priority. This is the reduced interleaving surrogate defined in the modeling chapter, not a reproduction of a vLLM scheduler. The control sequence specifies one normalized GPU frequency for each second of a 60-second horizon,

$$u_k = \frac{f_k - f_{\min}}{f_{\max} - f_{\min}} \in [0, 1]. \quad (46)$$

An aggregate state collects queued prefill work, active decode work, temperature, and the preceding frequency:

$$x_k = (p_k, d_k, T_k, f_{k-1}), \quad x_{k+1} = F_k(x_k, u_k, w_k). \quad (47)$$

The disturbance w_k contains the requests predicted to arrive during second k . Service and power in F_k interpolate the committed profile whose provenance is displayed with the result. The request-level simulator remains outside the optimizer and validates the resulting schedule after the solve.

The offline problem uses the complete nominal arrival forecast. It introduces a normalized service decision $\nu_k \in [\nu_{\min}, 1]$ and a nonnegative backlog variable B_{k+1} . If W_k is arriving work in seconds of highest-clock service, their fluid balance is relaxed to

$$\begin{aligned} & \underset{\nu_0, \dots, \nu_{59}, B_1, \dots, B_{60}}{\text{minimize}} && \sum_{k=0}^{59} [\alpha_P \nu_k + 20B_{k+1}] + 20B_{60} \\ & \text{subject to} && B_{k+1} \geq B_k + W_k - \Delta t \nu_k, \quad B_0 = 0, \\ & && B_{k+1} \geq 0, \quad \nu_{\min} \leq \nu_k \leq 1. \end{aligned} \quad (48)$$

The coefficient α_P is the slope of a linear interpolation between the lowest- and highest-clock normalized power values. The backlog term prices waiting throughout the horizon, while the additional terminal term discourages postponing work beyond second 60. This is a linear program, solved with HiGHS. The service decisions are mapped through the profiled service curve to continuous frequencies. Execution rounds each frequency downward to the nearest profiled clock, so the request-level validation includes the actuator’s finite action set.

This planning model deliberately omits request identities, phase-specific queues, clock slew, and the nonlinear thermal state. The detailed replay restores those variables and reports power, temperature, latency, and memory violations. The optimizer therefore supplies an offline plan from a tractable aggregate model, while the replay audits the assumptions used to obtain it.

The experiment asks what a one-shot frequency schedule gains from a perfect nominal workload forecast, and what it loses when that forecast is wrong. The same plan is replayed twice. The nominal replay uses the forecast supplied to the optimizer. In the shifted replay, the largest later ten-second work burst arrives twenty seconds earlier. No reoptimization occurs after either replay starts.

[Download every open-loop metric \(CSV\)](#)

HiGHS reports an optimal solution for the stated linear program, with objective 2,201.34 in its weighted model units. Under the nominal replay, the requests later selected for shifting have a mean time to first token of 5.83 seconds and a 95th percentile of 10.97 seconds. Moving only that burst into the plan’s low-clock interval raises these values to 11.49 and 19.57 seconds. At 30 seconds, the nominal replay has no queued request while the shifted replay has 25; the peak queue at the minimum clock rises from 4 to 25 requests.

The disturbance does not make every metric worse. Energy falls from 2,477 to 2,203 joules, and whole-trace mean time to first token falls from 7.12 to 6.77 seconds. Moving the burst changes which requests overlap later in the trace, so the aggregate mean hides the severe delay of the moved requests. This result shows that a schedule can conserve energy and improve an aggregate while being badly mistimed for a particular burst. The evidence should be read as a vector of aligned latency, queue, energy, and constraint metrics, not as a single verdict.

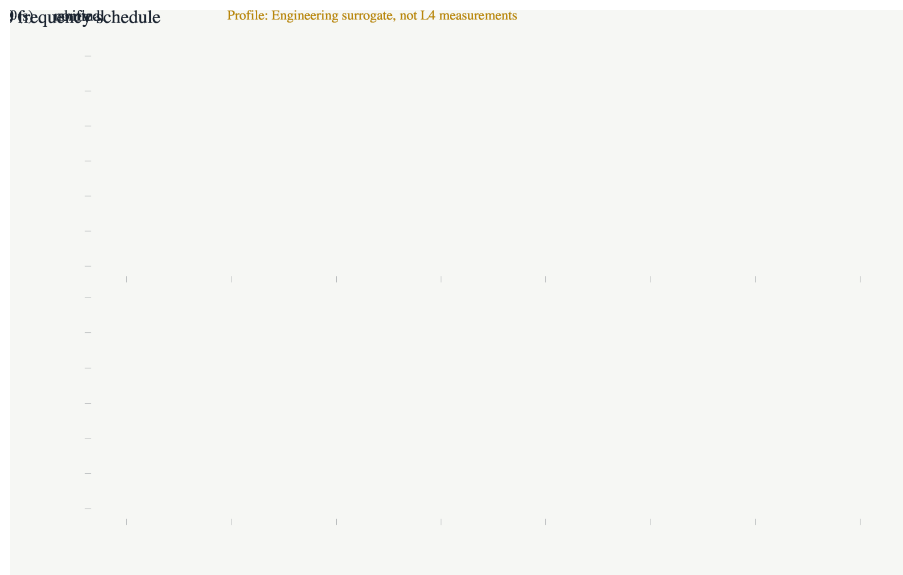


Figure 7: Static comparison of the nominal and shifted-burst replays. The online book adds playback and a controller selector.

Figure 8: Request-level metrics for the fixed offline schedule under its nominal forecast and the shifted-burst disturbance. The comparisons use the same requests, controller parameters, and profile provenance.

controller	nominal	shifted
metric		
mean_ttft_s	7.122	6.770
p95_ttft_s	12.443	19.345
matched_moved_burst_mean_ttft_s	11.833	11.492
matched_moved_burst_p95_ttft_s	10.968	19.572
energy_j	2477.409	2202.575
peak_queued_requests_at_minimum_clock	4.000	25.000
queued_requests_at_30_s	0.000	25.000
power_violation_w	0.000	0.000
thermal_violation_c	0.000	0.000

The metrics compare energy per output token, time to first token, time per output token, unfinished work, and power and thermal violations. The nominal run tests the optimized trajectory under its own assumptions. The shifted run tests sensitivity to one explicit forecast error. It does not establish robustness to arbitrary arrivals, model error, or hardware throttling. Closing that gap requires new information to alter future controls, which is the role of feedback and receding-horizon optimization. The current artifact is labeled as a pre-measurement engineering surrogate. The comparison tests this optimization mechanism but does not constitute a measured hardware result.

Example: Gradient Descent with Momentum as DOCP To connect this lens to familiar practice, including hyperparameter optimization, treat the learning rate and momentum (or their schedules) as controls. Rather than fixing them a priori, we can optimize them as part of a trajectory optimization. The optimizer itself becomes the dynamical system whose execution we shape to minimize final loss.

Program: gradient descent with momentum on a quadratic loss. We fit $\boldsymbol{\theta} \in \mathbb{R}^p$ to data $(\mathbf{A}, \mathbf{b})$ by minimizing

$$\ell(\boldsymbol{\theta}) = \frac{1}{2} \|\mathbf{A}\boldsymbol{\theta} - \mathbf{b}\|_2^2. \quad (49)$$

The program maintains parameters $\boldsymbol{\theta}_k$ and momentum $\mathbf{m}_k$. Each iteration does:

1. compute gradient $\mathbf{g}_k = \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}_k) = \mathbf{A}^\top (\mathbf{A}\boldsymbol{\theta}_k - \mathbf{b})$
2. update momentum $\mathbf{m}_{k+1} = \beta_k \mathbf{m}_k + \mathbf{g}_k$
3. update parameters $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha_k \mathbf{m}_{k+1}$

State, control, and transition. Define the state $\mathbf{x}_k = \begin{bmatrix} \boldsymbol{\theta}_k \\ \mathbf{m}_k \end{bmatrix} \in \mathbb{R}^{2p}$ and the control $\mathbf{u}_k = \begin{bmatrix} \alpha_k \\ \beta_k \end{bmatrix}$. One program step is

$$\Phi_k(\mathbf{x}_k, \mathbf{u}_k) = \begin{bmatrix} \boldsymbol{\theta}_k - \alpha_k (\beta_k \mathbf{m}_k + \mathbf{A}^\top (\mathbf{A}\boldsymbol{\theta}_k - \mathbf{b})) \\ \beta_k \mathbf{m}_k + \mathbf{A}^\top (\mathbf{A}\boldsymbol{\theta}_k - \mathbf{b}) \end{bmatrix}. \quad (50)$$

Executing the program for T iterations gives the trajectory

$$\mathbf{x}_{k+1} = \Phi_k(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, T-1, \quad \mathbf{x}_0 = \begin{bmatrix} \boldsymbol{\theta}_0 \\ \mathbf{m}_0 \end{bmatrix}. \quad (51)$$

Objective as a DOCP. Choose terminal cost $c_T(\mathbf{x}_T) = \ell(\boldsymbol{\theta}_T)$ and (optionally) stage costs $c_k(\mathbf{x}_k, \mathbf{u}_k) = \rho_\alpha \alpha_k^2 + \rho_\beta (\beta_k - \bar{\beta})^2$. The program-as-control problem is

$$\min_{\{\alpha_k, \beta_k\}} \ell(\boldsymbol{\theta}_T) + \sum_{k=0}^{T-1} (\rho_\alpha \alpha_k^2 + \rho_\beta (\beta_k - \bar{\beta})^2) \quad \text{s.t.} \quad \mathbf{x}_{k+1} = \Phi_k(\mathbf{x}_k, \mathbf{u}_k). \quad (52)$$

Backpropagation = reverse-time costate recursion. Because Φ_k is differentiable, reverse-mode AD computes $\nabla_{\mathbf{u}_{0:T-1}} (c_T + \sum c_k)$ by propagating a costate $\boldsymbol{\lambda}_k = \partial \mathcal{J} / \partial \mathbf{x}_k$ backward:

$$\boldsymbol{\lambda}_T = \nabla_{\mathbf{x}_T} c_T, \quad \boldsymbol{\lambda}_k = \nabla_{\mathbf{x}_k} c_k + (\nabla_{\mathbf{x}_k} \Phi_k)^\top \boldsymbol{\lambda}_{k+1}, \quad (53)$$

and the gradients with respect to controls are

$$\nabla_{\mathbf{u}_k} \mathcal{J} = \nabla_{\mathbf{u}_k} c_k + (\nabla_{\mathbf{u}_k} \Phi_k)^\top \boldsymbol{\lambda}_{k+1}. \quad (54)$$

Unrolling a tiny horizon ($T = 3$) to see the composition:

$$\begin{aligned}
\mathbf{x}_1 &= \Phi_0(\mathbf{x}_0, \mathbf{u}_0), \\
\mathbf{x}_2 &= \Phi_1(\mathbf{x}_1, \mathbf{u}_1), \\
\mathbf{x}_3 &= \Phi_2(\mathbf{x}_2, \mathbf{u}_2), \quad \mathcal{J} = c_T(\mathbf{x}_3) + \sum_{k=0}^2 c_k(\mathbf{x}_k, \mathbf{u}_k).
\end{aligned} \tag{55}$$

What if the program branches? Suppose we insert a “skip-small-gradients” branch

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha_k \mathbf{m}_{k+1} \mathbf{1}\{\|\mathbf{g}_k\| > \tau\}, \tag{56}$$

which is non-differentiable because of the indicator. The DOCP view still applies, but gradients are unreliable. Two practical paths: smooth the branch (e.g., replace $\mathbf{1}\{\cdot\}$ with $\sigma((\|\mathbf{g}_k\| - \tau)/\epsilon)$ for small ϵ) and use autodiff; or go derivative-free on $\{\alpha_k, \beta_k, \tau\}$ (e.g., SPSA or CMA-ES) while keeping the inner dynamics exact.

Variants: Lagrange and Mayer Problems The Bolza form is general enough to cover most situations, but two common special cases deserve mention:

- **Lagrange problem (no terminal cost)** If the objective only accumulates stage costs:

$$\min_{\mathbf{u}_{1:T-1}} \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t). \tag{57}$$

Example: *Energy minimization for a delivery drone*. The concern is total battery use, regardless of the final position.

- **Mayer problem (terminal cost only)** If the objective depends only on the final state:

$$\min_{\mathbf{u}_{1:T-1}} c_T(\mathbf{x}_T). \tag{58}$$

Example: *Satellite orbital transfer*. The only goal is to reach a specified orbit, no matter the fuel spent along the way.

These distinctions matter when deriving optimality conditions, but conceptually they fit in the same framework: the system evolves over time, and we choose controls to shape the trajectory.

Reducing to Mayer Form by State Augmentation Although Bolza, Lagrange, and Mayer problems look different, they are equivalent in expressive power. Any problem with running costs can be rewritten as a Mayer problem (one whose objective depends only on the final state) through a simple trick: **augment the state with a running sum of costs**.

The idea is straightforward. Introduce a new variable, y_t , that keeps track of the cumulative cost so far. At each step, we update this running sum along with the system state:

$$\tilde{\mathbf{x}}_{t+1} = \begin{pmatrix} \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) \\ y_t + c_t(\mathbf{x}_t, \mathbf{u}_t) \end{pmatrix}, \tag{59}$$

where $\tilde{\mathbf{x}}_t = (\mathbf{x}_t, y_t)$. The terminal cost then becomes:

$$\tilde{c}_T(\tilde{\mathbf{x}}_T) = c_T(\mathbf{x}_T) + y_T. \tag{60}$$

The overall effect is that the explicit sum $\sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t)$ disappears from the objective and is captured implicitly by the augmented state. This lets us write every optimal control problem in Mayer form.

This reduction serves two purposes. First, it often simplifies mathematical derivations, as we will see later when deriving necessary conditions. Second, it can streamline algorithmic implementation: instead of writing separate code paths for Mayer, Lagrange, and Bolza problems, we can reduce everything to one canonical form. That said, this unified approach is not always best in practice.

Specialized formulations can sometimes be more efficient computationally, especially when the running cost has simple structure.

The unifying theme is that a DOCP may look like a generic NLP on paper, but its structure matters. Ignoring that structure often leads to impractical solutions, whereas formulations that expose sparsity and respect temporal coupling allow modern solvers to scale effectively. In the following sections, we will examine how these choices play out in practice through single shooting, multiple shooting, and collocation methods, and why different formulations strike different trade-offs between robustness and computational effort.

Numerical Methods for Solving DOCPs

Before we discuss specific algorithms, it is useful to clarify the goal: we want to recast a discrete-time optimal control problem as a standard nonlinear program (NLP). Collect all decision variables (states, controls, and any auxiliary variables) into a single vector $\mathbf{z} \in \mathbb{R}^{n_z}$ and write

$$\begin{aligned} \min_{\mathbf{z} \in \mathbb{R}^{n_z}} \quad & F(\mathbf{z}) \\ \text{s.t.} \quad & G(\mathbf{z}) = 0, \\ & H(\mathbf{z}) \geq 0, \end{aligned} \tag{61}$$

with maps $F : \mathbb{R}^{n_z} \rightarrow \mathbb{R}$, $G : \mathbb{R}^{n_z} \rightarrow \mathbb{R}^{r_e}$, and $H : \mathbb{R}^{n_z} \rightarrow \mathbb{R}^{r_h}$. In optimal control, G typically encodes dynamics and boundary conditions, while H captures path and box constraints.

There are multiple ways to arrive at (and benefit from) this NLP:

- Simultaneous (direct transcription / full discretization): keep all states and controls as variables and impose the dynamics as equality constraints. This is straightforward and exposes sparsity, but the problem can be large unless solver-side techniques (e.g., condensing) are exploited.
- Sequential (recursive elimination / single shooting): eliminate states by forward propagation from the initial condition, leaving controls as the main decision variables. This reduces dimension and constraints, but can be sensitive to initialization and longer horizons.
- Multiple shooting: introduce state variables at segment boundaries and enforce continuity between simulated segments. This compromises between size and conditioning and is often more robust than pure single shooting.

The next sections work through these formulations, starting with simultaneous methods, then sequential methods, and finally multiple shooting, before discussing how generic NLP solvers and specialized algorithms leverage the resulting structure in practice.

Simultaneous Methods In the simultaneous (also called direct transcription or full discretization) approach, we keep the entire trajectory explicit and enforce the dynamics as equality constraints. Starting from the Bolza DOCP,

$$\min_{\{\mathbf{x}_t, \mathbf{u}_t\}} c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \quad \text{s.t.} \quad \mathbf{x}_{t+1} - \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) = 0, \quad t = 1, \dots, T-1, \tag{62}$$

collect all variables into a single vector

$$\mathbf{z} := \begin{bmatrix} \mathbf{x}_1^\top & \cdots & \mathbf{x}_T^\top & \mathbf{u}_1^\top & \cdots & \mathbf{u}_{T-1}^\top \end{bmatrix}^\top \in \mathbb{R}^{n_z}. \tag{63}$$

Path constraints typically apply only at selected times. Let $\mathcal{E}$ index additional equality constraints g_i and $\mathcal{I}$ index inequality constraints h_i . For each constraint i , define the set of time indices $K_i \subseteq \{1, \dots, T\}$ where it is enforced (e.g., terminal constraints use $K_i = \{T\}$). The simultaneous transcription is the NLP

$$\begin{aligned}
\min_{\mathbf{z}} \quad & F(\mathbf{z}) := c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \\
\text{s.t.} \quad & G(\mathbf{z}) = \begin{bmatrix} [g_i(\mathbf{x}_k, \mathbf{u}_k)]_{i \in \mathcal{C}, k \in K_i} \\ [\mathbf{x}_{t+1} - \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t)]_{t=1:T-1} \\ \mathbf{x}_1 - \mathbf{x}_{\text{init}} \end{bmatrix} = \mathbf{0}, \\
& H(\mathbf{z}) = [h_i(\mathbf{x}_k, \mathbf{u}_k)]_{i \in \mathcal{J}, k \in K_i} \geq \mathbf{0},
\end{aligned} \tag{64}$$

optionally with simple bounds $\mathbf{x}_{\text{lb}} \leq \mathbf{x}_t \leq \mathbf{x}_{\text{ub}}$ and $\mathbf{u}_{\text{lb}} \leq \mathbf{u}_t \leq \mathbf{u}_{\text{ub}}$ folded into H or provided to the solver separately. For notational convenience, some constraints may not depend on $\mathbf{u}_k$ at times in K_i ; the indexing still helps specify when each condition is active.

This direct transcription is attractive because it is faithful to the model and exposes sparsity. The Jacobian of G has a block bi-diagonal structure induced by the dynamics, and the KKT matrix is sparse and structured. These properties are exploited by interior-point and SQP methods. The trade-off is size: with state dimension n and control dimension m , the decision vector has $(Tn) + ((T-1) \cdot m)$ entries, and there are roughly $(T-1) \cdot n$ dynamic equalities plus any path and boundary conditions. Techniques such as partial or full condensing eliminate state variables to reduce the equality set (at the cost of denser matrices), while keeping states explicit preserves sparsity and often improves robustness on long horizons and in the presence of state constraints.

Compared to alternatives, simultaneous methods avoid the long nonlinear dependency chains of single shooting and make it easier to impose state/path constraints. They can, however, demand more memory and per-iteration linear algebra, so practical performance hinges on exploiting sparsity and good initialization.

The same logic applies when selecting an optimizer. For small-scale problems, it is common to rely on general-purpose routines such as those in `scipy.optimize.minimize`. Derivative-free methods like Nelder–Mead require no gradients but scale poorly as dimensionality increases. Quasi-Newton schemes such as BFGS work well for moderate dimensions and can approximate gradients by finite differences, while large-scale trajectory optimization often calls for gradient-based constrained solvers such as interior-point or sequential quadratic programming methods that can exploit sparse Jacobians and benefit from automatic differentiation. Stochastic techniques, including genetic algorithms, simulated annealing, or particle swarm optimization, occasionally appear when gradients are unavailable, but their cost grows rapidly with dimension and they are rarely competitive for structured optimal control problems.

Example: Nonlinear Cart-Pole Swing-Up A cart carries a rigid pendulum whose angle is measured from the upright vertical. The cart can accelerate horizontally, but no actuator applies torque directly at the pendulum joint. Starting from the stable downward configuration, the task is to move the base so that the pendulum arrives upright while the cart returns near the center of a finite rail.

Let the state be $\mathbf{x} = (p, v, \theta, \omega)$, where p and v are the cart position and velocity, and θ and ω are the pendulum angle and angular velocity. A commanded horizontal acceleration u produces the nonlinear dynamics

$$\dot{p} = v, \quad \dot{v} = u, \quad \dot{\theta} = \omega, \quad \dot{\omega} = \frac{g}{\ell} \sin \theta - \frac{u}{\ell} \cos \theta - b\omega. \tag{65}$$

The factor $-u \cos \theta / \ell$ identifies the action channel. Horizontal base motion couples into angular acceleration, and its sign and magnitude depend on the current configuration. A black-box optimizer could evaluate these equations without inspecting that term, but the term explains why the cart must first move away from its eventual resting position to build pendulum energy.

The numerical experiment uses a 4.5 s horizon with $N = 30$ zero-order-hold controls and a step size $h = 0.15$ s. Fourth-order Runge–Kutta integration defines the discrete map $\mathbf{x}_{k+1} = F_h(\mathbf{x}_k, u_k)$. Both numerical formulations solve the same problem:

$$\begin{aligned}
& \min_{\mathbf{x}_{0:N}, u_{0:N-1}} \quad h \sum_{k=0}^{N-1} \left[0.05p_k^2 + 0.01v_k^2 + 0.25(1 - \cos \theta_k) + 0.01\omega_k^2 + 0.004u_k^2 \right] \\
& \quad + 20p_N^2 + 5v_N^2 + 120(1 - \cos \theta_N) + 12\omega_N^2 \\
& \text{subject to} \quad \mathbf{x}_{k+1} = F_h(\mathbf{x}_k, u_k), \\
& \quad \mathbf{x}_0 = (0, 0, \pi, 0), \\
& \quad |p_k| \leq 2.4, \quad |v_k| \leq 4, \quad |\omega_k| \leq 12, \quad |u_k| \leq 8.
\end{aligned} \tag{66}$$

The periodic penalty $1 - \cos \theta$ assigns the same terminal cost to angles that differ by a full revolution. Position and velocity penalties still require the cart to finish near rest, so rotating the pole through the top is not enough by itself.

Direct transcription retains all 31 states and 30 controls. It therefore optimizes over 154 scalar variables and imposes 124 scalar equalities, including the initial condition and one four-dimensional dynamics equation per step. The equality Jacobian is block banded because the residual at step k depends only on $(\mathbf{x}_k, u_k, \mathbf{x}_{k+1})$.

The small demonstration below passes that Jacobian to SLSQP as a dense array. A large-scale direct solver would instead store and factor the same block-banded pattern sparsely. The formulation exposes sparsity, but exploiting it is a separate implementation choice.

Prediction before computation

Single shooting will reduce the decision vector from 154 variables to 30. Before examining the matched runs below, predict whether the smaller program must be easier to optimize. Identify which constraints become less explicit after the states are eliminated.

Sequential Methods The previous section showed how a discrete-time optimal control problem can be solved by treating all states and controls as decision variables and enforcing the dynamics as equality constraints. This produces a nonlinear program that can be passed to solvers such as `scipy.optimize.minimize` with the SLSQP method. For short horizons, this approach is straightforward and works well; the code stays close to the mathematical formulation.

It also has a real advantage: by keeping the states explicit and imposing the dynamics through constraints, we anchor the trajectory at multiple points. This extra structure helps stabilize the optimization, especially for long horizons where small deviations in early steps can otherwise propagate and cause the optimizer to drift or diverge. In that sense, this formulation is better conditioned and more robust than approaches that treat the dynamics implicitly.

The drawback is scale. As the horizon grows, the number of variables and constraints grows with it, and all are coupled by the dynamics. Each iteration of a sequential quadratic programming (SQP) or interior-point method requires building and factorizing large Jacobians and Hessians. These methods have been embedded in reinforcement learning and differentiable programming pipelines, through implicit layers or differentiable convex solvers, but the cost is significant. They remain serial, rely on repeated linear algebra factorizations, and are difficult to parallelize efficiently. When thousands of such problems must be solved inside a learning loop, the overhead becomes prohibitive.

This motivates an alternative that aligns with the computational model of machine learning. For deterministic dynamics, the equality constraints can be eliminated by making the states implicit. Instead of solving for both states and controls, we fix the initial state and roll the system forward under a candidate control sequence. State constraints can remain, but they become nonlinear functions of the entire preceding control sequence. This is the essence of **single shooting**.

The term “shooting” comes from the idea of *aiming and firing* a trajectory from the initial state: you pick a control sequence, integrate (or step) the system forward, and see where it lands. If the final state misses the target, you adjust the controls and try again: like adjusting the angle of a shot until it hits the mark. It is called **single** shooting because we compute the entire trajectory in one pass from the starting point, without breaking it into segments. Later, we will contrast this with **multiple**

shooting, where the horizon is divided into smaller arcs that are optimized jointly to improve stability and conditioning.

The analogy with deep learning is also immediate: the control sequence plays the role of parameters, the rollout is a forward pass, and the cost is a scalar loss. Gradients can be obtained with reverse-mode automatic differentiation. In the single shooting formulation of the DOCP, the constrained program

$$\min_{\mathbf{x}_{1:T}, \mathbf{u}_{1:T-1}} J(\mathbf{x}_{1:T}, \mathbf{u}_{1:T-1}) \quad \text{s.t.} \quad \mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) \quad (67)$$

collapses to

$$\min_{\mathbf{u}_{1:T-1}} c_T(\phi_T(\mathbf{u}, \mathbf{x}_1)) + \sum_{t=1}^{T-1} c_t(\phi_t(\mathbf{u}, \mathbf{x}_1), \mathbf{u}_t), \quad \text{s.t.} \quad \mathbf{h}_t(\phi_t(\mathbf{u}, \mathbf{x}_1), \mathbf{u}_t) \geq 0, \quad \mathbf{u}_{\text{lb}} \leq \mathbf{u}_t \leq \mathbf{u}_{\text{ub}}. \quad (68)$$

Here ϕ_t denotes the state reached at time t by recursively applying the dynamics to the previous state and current control. This recursion can be written as

$$\phi_{t+1}(\mathbf{u}, \mathbf{x}_1) = \mathbf{f}_t(\phi_t(\mathbf{u}, \mathbf{x}_1), \mathbf{u}_t), \quad \phi_1 = \mathbf{x}_1. \quad (69)$$

Concretely, here is JAX-style pseudocode for defining `phi(u, x_0, t)` using `jax.lax.scan` with a zero-based time index:

```
def phi(u_seq, x0, t):
    """Return \phi_t(u, x0) with 0-based t (\phi_0 = x0).

    u_seq: controls of length T (or T-1); only first t entries are used
    x0: initial state at time 0
    t: integer >= 0
    """
    if t <= 0:
        return x0

    def step(carry, u):
        x, t_idx = carry
        x_next = f(x, u, t_idx)
        return (x_next, t_idx + 1), None

    (x_t, _), _ = lax.scan(step, (x0, 0), u_seq[:t])
    return x_t
```

The pattern mirrors an RNN unroll: starting from an initial state ($\mathbf{x}_1^*$) and a sequence of controls ($\mathbf{u}_{1:T-1}^*$), we propagate forward through the dynamics, updating the state at each step and accumulating cost along the way. This structural similarity is why single shooting often feels natural to practitioners with a deep learning background: the rollout is a forward pass, and gradients propagate backward through time exactly as in backpropagation through an RNN.

Algorithmically:

Algorithm 1: Single Shooting: Forward Unroll

Inputs: Initial state $\mathbf{x}_1$, horizon T , control bounds $\mathbf{u}_{\text{lb}}, \mathbf{u}_{\text{ub}}$, dynamics $\mathbf{f}_t$, costs c_t

Output: Optimal control sequence $\mathbf{u}_{1:T-1}^*$

1. Initialize $\mathbf{u}_{1:T-1}$ within bounds
2. Define `ComputeTrajectoryAndCost(u, x1)`: